

Geometric Approach to Symmetric-Key Cryptanalysis

对称密码分析的几何方法

Kai Hu

School of Cyber Science and Technology
Shandong University
kai.hu@sdu.edu.cn



密码学高端系列培训之十四 · 中国密码学会
北京, 2026 年 8 月 25 日



Part I

Distinguisher of ciphers: property and approximation

- 1 The task in symmetric-key cryptanalysis
- 2 Revisit linear cryptanalysis with geometric approach
- 3 Approximation with trails

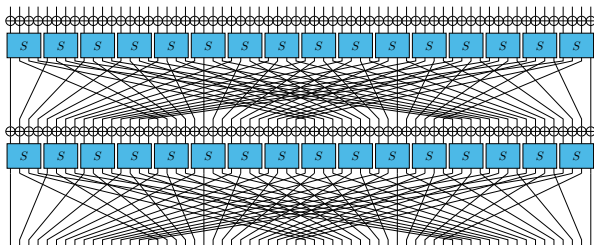
Block cipher

A block cipher maps **fixed-size** input blocks to fixed-size output blocks.

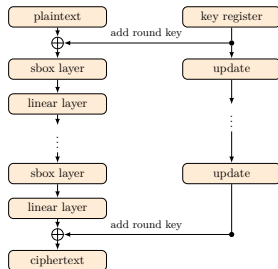
Structurally, it is typically built upon three core components:

- **Non-linear layers** (e.g., Sboxes for confusion)
- **Linear layers** (e.g., Bit-permutations for diffusion)
- **Key mixing layer** (e.g., Key-XORs)

Example: PRESENT block cipher



Round Structure



Overall Framework

[Bogdanov et al., CHES 2007]

Security foundations: public-key cryptosystems vs. block ciphers

Public-key cryptosystems (e.g., RSA)

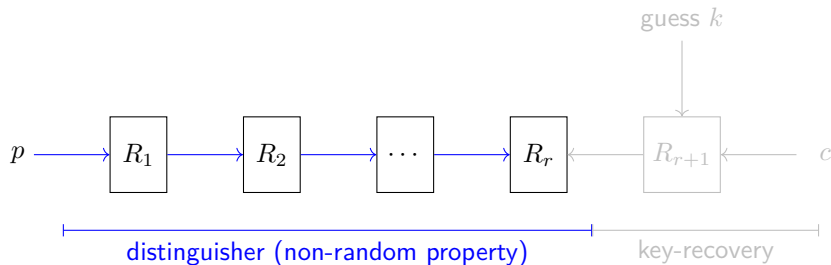
- Security relies on the **hardness** of specific mathematical problems.
- Example: **Integer factorization**.
- *Status*: No classical polynomial-time algorithm is **known**, but impossibility is **not proven**.

Block ciphers (e.g., AES, DES)

- **No underlying mathematical hard problem** has been identified.
- Security is based on **heuristic arguments** and **extensive cryptanalysis**.
- *Principle*: “Security through scrutiny” — if no efficient attack is found after years of analysis by experts, we consider it secure.

Cryptanalysis of block ciphers

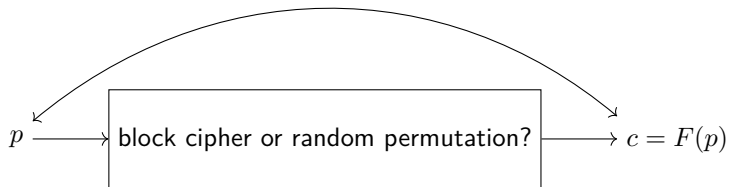
- Attackers try to **distinguish** it from a **random permutation** (i.e., an ideal cipher) using a **data complexity** below 2^n , i.e. without querying the full codebook of an n -bit block cipher.
- Designers try to prove that it cannot be **distinguished** within this bound.



Linear distinguisher

- **Motivation:** the input and output might be linearly related.

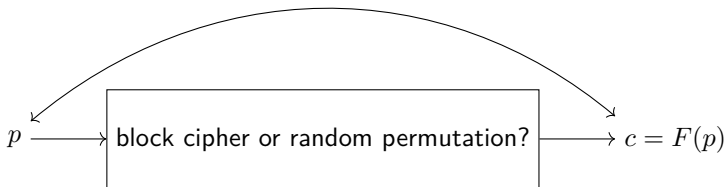
$$\varepsilon = \text{Pr}_{p \in \mathbb{F}_2^n} [p_{i_1} \oplus p_{i_2} \oplus \cdots \oplus p_{i_s} = c_{j_1} \oplus c_{j_2} \oplus \cdots \oplus c_{j_t}] - \frac{1}{2}$$



Linear distinguisher

- **Motivation:** the input and output might be linearly related.

$$\varepsilon = \text{Pro}_{p \in \mathbb{F}_2^n} \left[p_{i_1} \oplus p_{i_2} \oplus \cdots \oplus p_{i_s} = c_{j_1} \oplus c_{j_2} \oplus \cdots \oplus c_{j_t} \right] - \frac{1}{2}$$



- For random: $|\varepsilon| \approx 0$
- Mask: $\gamma \in \mathbb{F}_2^n$

$$\gamma^\top x := \bigoplus_{i=0}^{n-1} \gamma_i \cdot x_i = \bigoplus_{0 \leq i < n, \gamma_i = 1} x_i$$

- Bias:

$$\text{Bias}_F(\gamma, \eta) = \text{Pro}_{p \in \mathbb{F}_2^n} [\gamma^\top p = \eta^\top F(p)] - \frac{1}{2}$$

[Matsui, EUROCRYPT 1993]

- Correlation of a variable $x \in \mathbb{F}_2$:

$$\begin{aligned}\text{Cor}_x &= \text{Pro}_x[x = 0] - \text{Pro}_x[x = 1] \\ &= \text{Pro}_x[x = 0] - (1 - \text{Pro}_x[x = 0]) = 2\text{Bias}_x\end{aligned}$$

Correlation

- Correlation of a variable $x \in \mathbb{F}_2$:

$$\begin{aligned}\text{Cor}_x &= \text{Pro}_x[x = 0] - \text{Pro}_x[x = 1] \\ &= \text{Pro}_x[x = 0] - (1 - \text{Pro}_x[x = 0]) = 2\text{Bias}_x\end{aligned}$$

- Correlation represents the same thing as the bias, **but** is usually **more convenient** than bias:

Correlation

- Correlation of a variable $x \in \mathbb{F}_2$:

$$\begin{aligned}\text{Cor}_x &= \text{Pro}_x[x = 0] - \text{Pro}_x[x = 1] \\ &= \text{Pro}_x[x = 0] - (1 - \text{Pro}_x[x = 0]) = 2\text{Bias}_x\end{aligned}$$

- Correlation represents the same thing as the bias, **but** is usually **more convenient** than bias:
 - a more convenient sum expression (see later)

$$\text{Cor}_F(\gamma, \eta) = 2^{-n} \sum_{p \in \mathbb{F}_2^n} (-1)^{\gamma^\top p \oplus \eta^\top F(p)}$$

- easier for Piling-up lemma: Let $x_0, x_1 \in \mathbb{F}_2$ be independent

$$\text{Cor}_{x_0 \oplus x_1} = \text{Cor}_{x_0} \cdot \text{Cor}_{x_1}$$

$$(\text{Bias}_{x_0 \oplus x_1} = 2 \cdot \text{Bias}_{x_0} \cdot \text{Bias}_{x_1})$$

Perform a linear distinguisher

- Suppose a block cipher has a linear distinguisher with a correlation:

$$\text{Cor}_F(\gamma, \eta) = \theta$$

Perform a linear distinguisher

- Suppose a block cipher has a linear distinguisher with a correlation:

$$\text{Cor}_F(\gamma, \eta) = \theta$$

- Collect N independent plaintext-ciphertext pairs $\{(p_i, F(p_i))\}$, construct

$$T = \frac{1}{N} \sum_{0 \leq i < N} (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)}$$

- Block cipher:

- Let $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)}$; $E(X_i) = \theta$ and $E(X_i^2) = 1$.
- $E(T) = \theta$, $\text{Var}(T) = \frac{1-\theta^2}{N} \approx \frac{1}{N}$ (assume θ is small).
- When N is large, $T \sim \mathcal{N}(\theta, \frac{1}{N})$.

Perform a linear distinguisher

- Suppose a block cipher has a linear distinguisher with a correlation:

$$\text{Cor}_F(\gamma, \eta) = \theta$$

- Collect N independent plaintext-ciphertext pairs $\{(p_i, F(p_i))\}$, construct

$$T = \frac{1}{N} \sum_{0 \leq i < N} (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)}$$

- Block cipher:

- Let $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)}$; $E(X_i) = \theta$ and $E(X_i^2) = 1$.
- $E(T) = \theta$, $\text{Var}(T) = \frac{1-\theta^2}{N} \approx \frac{1}{N}$ (assume θ is small).
- When N is large, $T \sim \mathcal{N}(\theta, \frac{1}{N})$.

- Random:

- $T \sim \mathcal{N}(0, \frac{1}{N})$.



The statistics of T (1): one sample

- Each sample is one random ± 1 : $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)} \in \{+1, -1\}$.



The statistics of $T(1)$: one sample

- Each sample is one random ± 1 : $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)} \in \{+1, -1\}$.
- Its mean:

$$\begin{aligned} E(X_i) &= 1 \cdot P(X_i = +1) + (-1) \cdot P(X_i = -1) \\ &= P(X_i = +1) - P(X_i = -1) \end{aligned}$$



The statistics of $T(1)$: one sample

- Each sample is one random ± 1 : $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)} \in \{+1, -1\}$.

- Its mean:

$$\begin{aligned} E(X_i) &= 1 \cdot P(X_i = +1) + (-1) \cdot P(X_i = -1) \\ &= P(X_i = +1) - P(X_i = -1) \end{aligned}$$

- Connect the sign to the mask value: write $b_i = \gamma^\top p_i \oplus \eta^\top F(p_i)$, then

$$X_i = +1 \iff b_i = 0, \quad X_i = -1 \iff b_i = 1$$

so

$$P(X_i = +1) - P(X_i = -1) = P(b_i = 0) - P(b_i = 1) = \text{Cor}_F(\gamma, \eta) = \theta$$

(the last step is the **definition** of the correlation.)



The statistics of $T(1)$: one sample

- Each sample is one random ± 1 : $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)} \in \{+1, -1\}$.

- Its mean:

$$\begin{aligned} E(X_i) &= 1 \cdot P(X_i = +1) + (-1) \cdot P(X_i = -1) \\ &= P(X_i = +1) - P(X_i = -1) \end{aligned}$$

- Connect the sign to the mask value: write $b_i = \gamma^\top p_i \oplus \eta^\top F(p_i)$, then

$$X_i = +1 \iff b_i = 0, \quad X_i = -1 \iff b_i = 1$$

so

$$P(X_i = +1) - P(X_i = -1) = P(b_i = 0) - P(b_i = 1) = \text{Cor}_F(\gamma, \eta) = \theta$$

(the last step is the **definition** of the correlation.)

- Its variance, from $\text{Var}(X) = E(X^2) - E(X)^2$; since $X_i^2 = 1$ always:
 $\text{Var}(X_i) = 1 - \theta^2$.



The statistics of $T(1)$: one sample

- Each sample is one random ± 1 : $X_i = (-1)^{\gamma^\top p_i \oplus \eta^\top F(p_i)} \in \{+1, -1\}$.

- Its mean:

$$\begin{aligned} E(X_i) &= 1 \cdot P(X_i = +1) + (-1) \cdot P(X_i = -1) \\ &= P(X_i = +1) - P(X_i = -1) \end{aligned}$$

- Connect the sign to the mask value: write $b_i = \gamma^\top p_i \oplus \eta^\top F(p_i)$, then

$$X_i = +1 \iff b_i = 0, \quad X_i = -1 \iff b_i = 1$$

so

$$P(X_i = +1) - P(X_i = -1) = P(b_i = 0) - P(b_i = 1) = \text{Cor}_F(\gamma, \eta) = \theta$$

(the last step is the **definition** of the correlation.)

- Its variance, from $\text{Var}(X) = E(X^2) - E(X)^2$; since $X_i^2 = 1$ always:
 $\text{Var}(X_i) = 1 - \theta^2$.
- Check:** $\theta = 0$ (random) gives $\text{Var}(X_i) = 1$ — a fair ± 1 .



The statistics of T (2): averaging N samples

- The statistic is the average of N independent samples:

$$T = \frac{1}{N} \sum_{i=0}^{N-1} X_i$$



The statistics of T (2): averaging N samples

- The statistic is the average of N independent samples:

$$T = \frac{1}{N} \sum_{i=0}^{N-1} X_i$$

- Mean (linearity of expectation): $E(T) = \frac{1}{N} \cdot N \cdot \theta = \theta$.



The statistics of T (2): averaging N samples

- The statistic is the average of N independent samples:

$$T = \frac{1}{N} \sum_{i=0}^{N-1} X_i$$

- Mean (linearity of expectation): $E(T) = \frac{1}{N} \cdot N \cdot \theta = \theta$.
- Variance (independent $\Rightarrow$ variances add; scaling by $1/N$ gives $1/N^2$):

$$\text{Var}(T) = \frac{1}{N^2} \cdot N \cdot (1 - \theta^2) = \frac{1 - \theta^2}{N} \stackrel{\theta \ll 1}{\approx} \frac{1}{N}$$



The statistics of T (2): averaging N samples

- The statistic is the average of N independent samples:

$$T = \frac{1}{N} \sum_{i=0}^{N-1} X_i$$

- Mean (linearity of expectation): $E(T) = \frac{1}{N} \cdot N \cdot \theta = \theta$.
- Variance (independent $\Rightarrow$ variances add; scaling by $1/N$ gives $1/N^2$):

$$\text{Var}(T) = \frac{1}{N^2} \cdot N \cdot (1 - \theta^2) = \frac{1 - \theta^2}{N} \stackrel{\theta \ll 1}{\approx} \frac{1}{N}$$

- Central limit theorem (N large): T is approximately normal,

$$T \sim \mathcal{N}\left(\theta, \frac{1-\theta^2}{N}\right), \quad \frac{1-\theta^2}{N} \approx \frac{1}{N} \quad (\theta \ll 1)$$

so $T \sim \mathcal{N}\left(\theta, \frac{1}{N}\right)$; for a random permutation ($\theta = 0$): $T \sim \mathcal{N}\left(0, \frac{1}{N}\right)$.

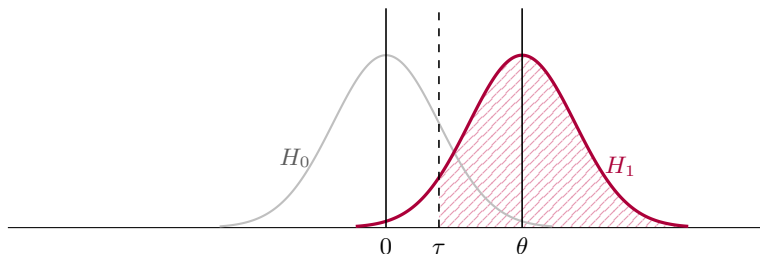
- **This is why** N shows up in the data-complexity formula.

Hypothesis testing

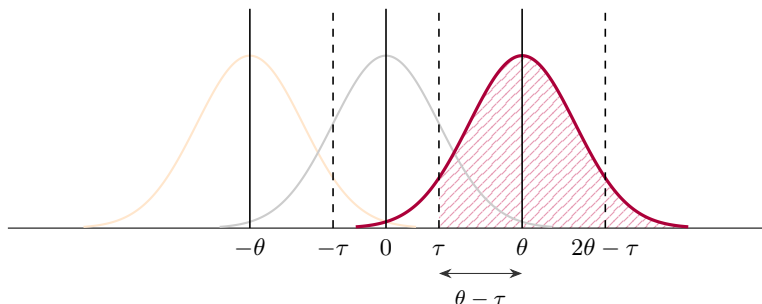
$$H_0 : T \sim \mathcal{N}(0, \frac{1}{N}); \quad H_1 : T \sim \mathcal{N}(\theta, \frac{1}{N}).$$

	decide H_0	decide H_1
truth H_1	false negative	true positive
truth H_0	true negative	false positive

rows: truth; columns: our decision



Distinguish two normal distributions (1)

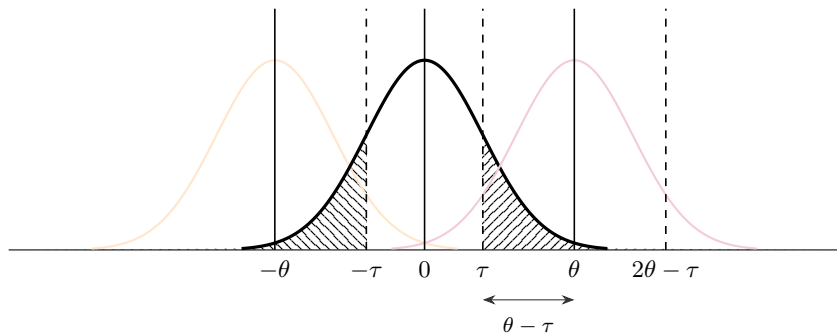


- Assume $\theta > 0$,
 - Decision rule: fix a threshold τ and declare **cipher** if $|T| > \tau$.
 - True positive:

$$P_S = \Phi(\sqrt{N}(\theta - \tau)) + \Phi(-\sqrt{N}(\theta + \tau)) \approx \Phi(\sqrt{N}(\theta - \tau))$$

- Second term: $\Pr[T < -\tau]$, of order $\Phi(-\sqrt{N}\theta)$ — negligible.

Distinguish two normal distributions (2)



- False positive: $P_F = 2(1 - \Phi(\sqrt{N}\tau))$.
- Cancel τ : $\theta - \frac{\Phi^{-1}(P_S)}{\sqrt{N}} = \frac{\Phi^{-1}(1 - \frac{P_F}{2})}{\sqrt{N}}$.
- Data complexity is determined by fixing P_S and P_F .

$$N = \left(\frac{\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})}{\theta} \right)^2$$



How P_S is computed

- Cipher: $T \sim \mathcal{N}(\theta, \frac{1}{N})$. Standardize: $Z = \sqrt{N}(T - \theta) \sim \mathcal{N}(0, 1)$.



How P_S is computed

- Cipher: $T \sim \mathcal{N}(\theta, \frac{1}{N})$. Standardize: $Z = \sqrt{N}(T - \theta) \sim \mathcal{N}(0, 1)$.
- Two disjoint tails: $P_S = \Pr[|T| > \tau] = \Pr[T > \tau] + \Pr[T < -\tau]$,

$$\Pr[T > \tau] = \Pr[Z > \sqrt{N}(\tau - \theta)] = \Phi(\sqrt{N}(\theta - \tau)),$$

$$\Pr[T < -\tau] = \Pr[Z < -\sqrt{N}(\tau + \theta)] = \Phi(-\sqrt{N}(\theta + \tau)).$$



How P_S is computed

- Cipher: $T \sim \mathcal{N}(\theta, \frac{1}{N})$. Standardize: $Z = \sqrt{N}(T - \theta) \sim \mathcal{N}(0, 1)$.
- Two disjoint tails: $P_S = \Pr[|T| > \tau] = \Pr[T > \tau] + \Pr[T < -\tau]$,

$$\Pr[T > \tau] = \Pr[Z > \sqrt{N}(\tau - \theta)] = \Phi(\sqrt{N}(\theta - \tau)),$$

$$\Pr[T < -\tau] = \Pr[Z < -\sqrt{N}(\tau + \theta)] = \Phi(-\sqrt{N}(\theta + \tau)).$$

- Hence

$$P_S = \Phi(\sqrt{N}(\theta - \tau)) + \Phi(-\sqrt{N}(\theta + \tau)),$$

with the second term $\leq \Phi(-\sqrt{N}\theta) \rightarrow 0$ (negligible).



How P_F is computed

- Random permutation: $T \sim \mathcal{N}(0, \frac{1}{N})$. Standardize: $Z = \sqrt{N} T \sim \mathcal{N}(0, 1)$.



How P_F is computed

- Random permutation: $T \sim \mathcal{N}(0, \frac{1}{N})$. Standardize: $Z = \sqrt{N} T \sim \mathcal{N}(0, 1)$.
- Two-sided, symmetric around 0:

$$P_F = \Pr[|T| > \tau] = 2 \Pr[T > \tau].$$



How P_F is computed

- Random permutation: $T \sim \mathcal{N}(0, \frac{1}{N})$. Standardize: $Z = \sqrt{N} T \sim \mathcal{N}(0, 1)$.
- Two-sided, symmetric around 0:

$$P_F = \Pr[|T| > \tau] = 2 \Pr[T > \tau].$$

- Standardize:

$$P_F = 2 \Pr[Z > \sqrt{N} \tau] = 2(1 - \Phi(\sqrt{N} \tau)).$$



Solving for N

- Fix target P_S, P_F ; eliminate the threshold τ .



Solving for N

- Fix target P_S, P_F ; eliminate the threshold τ .
- From $P_S \approx \Phi(\sqrt{N}(\theta - \tau))$:

$$\sqrt{N}(\theta - \tau) = \Phi^{-1}(P_S) \implies \tau = \theta - \frac{\Phi^{-1}(P_S)}{\sqrt{N}}.$$



Solving for N

- Fix target P_S, P_F ; eliminate the threshold τ .
- From $P_S \approx \Phi(\sqrt{N}(\theta - \tau))$:

$$\sqrt{N}(\theta - \tau) = \Phi^{-1}(P_S) \implies \tau = \theta - \frac{\Phi^{-1}(P_S)}{\sqrt{N}}.$$

- From $P_F = 2(1 - \Phi(\sqrt{N}\tau))$:

$$\Phi(\sqrt{N}\tau) = 1 - \frac{P_F}{2} \implies \tau = \frac{\Phi^{-1}(1 - \frac{P_F}{2})}{\sqrt{N}}.$$



Solving for N

- Fix target P_S, P_F ; eliminate the threshold τ .
- From $P_S \approx \Phi(\sqrt{N}(\theta - \tau))$:

$$\sqrt{N}(\theta - \tau) = \Phi^{-1}(P_S) \implies \tau = \theta - \frac{\Phi^{-1}(P_S)}{\sqrt{N}}.$$

- From $P_F = 2(1 - \Phi(\sqrt{N}\tau))$:

$$\Phi(\sqrt{N}\tau) = 1 - \frac{P_F}{2} \implies \tau = \frac{\Phi^{-1}(1 - \frac{P_F}{2})}{\sqrt{N}}.$$

- Equate the two τ 's and solve:

$$\theta - \frac{\Phi^{-1}(P_S)}{\sqrt{N}} = \frac{\Phi^{-1}(1 - \frac{P_F}{2})}{\sqrt{N}} \implies N = \left(\frac{\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})}{\theta} \right)^2.$$

Contents

- 1 The task in symmetric-key cryptanalysis
- 2 Revisit linear cryptanalysis with geometric approach
- 3 Approximation with trails

From intuition-based approaches to systematic methods

- Different attacks have been derived from various intuitive ideas.
- Grasping the essence of these intuitions is difficult.
- Different cryptanalytic techniques appear to lack a strong theoretical connection.
- This makes it difficult to propose new cryptanalytic attacks.

From intuition-based approaches to systematic methods

- Different attacks have been derived from various intuitive ideas.
- Grasping the essence of these intuitions is difficult.
- Different cryptanalytic techniques appear to lack a strong theoretical connection.
- This makes it difficult to propose new cryptanalytic attacks.
- **The geometric approach** is a theoretical framework proposed by Dr. Tim Beyne to unify all cryptanalytic attacks under one coherent model.

[Beyne, ASIACRYPT 2021] [Beyne, PhD thesis, KU Leuven 2023]

Free vector spaces

- Free Vector Space: X is a **finite** set and $\mathbb{K}$ is a field,

$$\mathbb{K}[X] = \left\{ \sum_{x \in X} c_x \delta_x \mid c_x \in \mathbb{K} \right\}$$

- $x \in X \implies$ unique basis vector δ_x
- Example: $X = \{a, b\} \implies \mathbb{F}_2[X] = \{0, \delta_a, \delta_b, \delta_a + \delta_b\}$

Free vector spaces

- Free Vector Space: X is a **finite** set and $\mathbb{K}$ is a field,

$$\mathbb{K}[X] = \left\{ \sum_{x \in X} c_x \delta_x \mid c_x \in \mathbb{K} \right\}$$

- $x \in X \implies$ unique basis vector δ_x
- Example: $X = \{a, b\} \implies \mathbb{F}_2[X] = \{0, \delta_a, \delta_b, \delta_a + \delta_b\}$
- $\mathbb{K}[X]$ is a vector space,
 - Operations component-wise on c_x :

$$\sum_{x \in X} c_x \delta_x + \sum_{x \in X} d_x \delta_x = \sum_{x \in X} (c_x + d_x) \delta_x, \quad k \sum_{x \in X} c_x \delta_x = \sum_{x \in X} k c_x \delta_x$$

- $\dim(\mathbb{K}[X]) = |X|$, and $\{\delta_x \mid x \in X\}$ is a natural **standard basis**.

Lifting non-linear functions to linear operators

- Intuition: study a nonlinear $F : X \rightarrow X$ using linear algebra.
- Define a linear map,

$$T^F : \mathbb{K}[X] \longrightarrow \mathbb{K}[X]; \quad \delta_x \mapsto \delta_{F(x)}$$

- Thus,

$$T^F \left(\sum_{x \in X} c_x \delta_x \right) = \sum_{x \in X} c_x T^F(\delta_x) = \sum_{x \in X} c_x \delta_{F(x)}$$

The dual space

- A **linear functional** on a $\mathbb{K}$ -vector space V is a linear map $\varphi : V \rightarrow \mathbb{K}$.

The dual space

- A **linear functional** on a $\mathbb{K}$ -vector space V is a linear map $\varphi : V \rightarrow \mathbb{K}$.
- All linear functionals on V form a vector space (add and scale pointwise) — the **dual space** $V^* = \text{Hom}(V, \mathbb{K})$.

The dual space

- A **linear functional** on a $\mathbb{K}$ -vector space V is a linear map $\varphi : V \rightarrow \mathbb{K}$.
- All linear functionals on V form a vector space (add and scale pointwise) — the **dual space** $V^* = \text{Hom}(V, \mathbb{K})$.
- For a basis $(\beta_u, u \in X)$ of $V = \mathbb{K}[X]$, there is a unique **dual basis** $(\beta^u, u \in X)$ of V^* , fixed by

$$\beta^u(\beta_v) = \delta_{u,v} := \begin{cases} 1 & u = v \\ 0 & u \neq v. \end{cases}$$

- Subscript: a basis vector. **Superscript: the dual functional.**

Functionals are functions $X \rightarrow \mathbb{K}$

- A functional $f : \mathbb{K}[X] \rightarrow \mathbb{K}$ is determined by its values on the standard basis $(\delta_x, x \in X)$: for any $w = \sum_x c_x \delta_x$,

$$f(w) = \sum_x c_x f(\delta_x).$$

Functionals are functions $X \rightarrow \mathbb{K}$

- A functional $f : \mathbb{K}[X] \rightarrow \mathbb{K}$ is determined by its values on the standard basis $(\delta_x, x \in X)$: for any $w = \sum_x c_x \delta_x$,

$$f(w) = \sum_x c_x f(\delta_x).$$

- Hence f is equivalently a function $X \rightarrow \mathbb{K}$, namely $x \mapsto f(\delta_x)$:

$$\text{Hom}(\mathbb{K}[X], \mathbb{K}) \cong \mathbb{K}^X.$$

Functionals are functions $X \rightarrow \mathbb{K}$

- A functional $f : \mathbb{K}[X] \rightarrow \mathbb{K}$ is determined by its values on the standard basis $(\delta_x, x \in X)$: for any $w = \sum_x c_x \delta_x$,

$$f(w) = \sum_x c_x f(\delta_x).$$

- Hence f is equivalently a function $X \rightarrow \mathbb{K}$, namely $x \mapsto f(\delta_x)$:

$$\text{Hom}(\mathbb{K}[X], \mathbb{K}) \cong \mathbb{K}^X.$$

- So each dual functional β^u becomes a function on X , written

$$\beta^u(x) := \beta^u(\delta_x).$$

Dual basis

- The dual basis $(\beta^u, u \in X)$ **reads off coordinates**. Write any vector w as a linear combination:

$$w = \sum_{u \in X} c_u \beta_u.$$

- The dual basis $(\beta^u, u \in X)$ **reads off coordinates**. Write any vector w as a linear combination:

$$w = \sum_{u \in X} c_u \beta_u.$$

- Apply β^v to both sides; by $\beta^v(\beta_u) = \delta_{v,u}$, only the v -th term survives:

$$\beta^v(w) = c_v.$$

- The dual basis $(\beta^u, u \in X)$ **reads off coordinates**. Write any vector w as a linear combination:

$$w = \sum_{u \in X} c_u \beta_u.$$

- Apply β^v to both sides; by $\beta^v(\beta_u) = \delta_{v,u}$, only the v -th term survives:

$$\beta^v(w) = c_v.$$

- Hence

$$w = \sum_{u \in X} \beta^u(w) \beta_u,$$

i.e. the u -th coordinate of w under $(\beta_u, u \in X)$ is the number $\beta^u(w)$.

Matrix entries via the dual basis

- The matrix of T^F under (β_u) is built **column by column**: its u -th column holds the coordinates of $T^F \beta_u$. So A^F is **defined** by

$$T^F \beta_u = \sum_{v \in X} A^F[v, u] \beta_v. \quad (1)$$

Matrix entries via the dual basis

- The matrix of T^F under (β_u) is built **column by column**: its u -th column holds the coordinates of $T^F \beta_u$. So A^F is **defined** by

$$T^F \beta_u = \sum_{v \in X} A^F[v, u] \beta_v. \quad (1)$$

- Reading off the coordinates of $w = T^F \beta_u$ with the previous slide:

$$T^F \beta_u = \sum_{v \in X} \beta^v(T^F \beta_u) \beta_v. \quad (2)$$

Matrix entries via the dual basis

- The matrix of T^F under (β_u) is built **column by column**: its u -th column holds the coordinates of $T^F \beta_u$. So A^F is **defined** by

$$T^F \beta_u = \sum_{v \in X} A^F[v, u] \beta_v. \quad (1)$$

- Reading off the coordinates of $w = T^F \beta_u$ with the previous slide:

$$T^F \beta_u = \sum_{v \in X} \beta^v(T^F \beta_u) \beta_v. \quad (2)$$

- Coordinates with respect to a basis are **unique**, so (1) and (2) must agree coefficient by coefficient:

$$A^F[v, u] = \beta^v(T^F \beta_u).$$



How to compute $A^F[v, u] = \beta^v(T^F \beta_u)$

- Goal: turn the boxed formula into a sum we can evaluate.



How to compute $A^F[v, u] = \beta^v(T^F \beta_u)$

- Goal: turn the boxed formula into a sum we can evaluate.
- Step 1 — expand β_u in the standard basis (δ_x) :

$$\beta_u = \sum_{x \in X} \beta_u[x] \delta_x.$$



How to compute $A^F[v, u] = \beta^v(T^F \beta_u)$

- Goal: turn the boxed formula into a sum we can evaluate.
- Step 1 — expand β_u in the standard basis (δ_x):

$$\beta_u = \sum_{x \in X} \beta_u[x] \delta_x.$$

- Step 2 — T^F is linear and sends $\delta_x \mapsto \delta_{F(x)}$:

$$T^F \beta_u = \sum_{x \in X} \beta_u[x] \delta_{F(x)}.$$



How to compute $A^F[v, u] = \beta^v(T^F \beta_u)$

- Goal: turn the boxed formula into a sum we can evaluate.
- Step 1 — expand β_u in the standard basis (δ_x) :

$$\beta_u = \sum_{x \in X} \beta_u[x] \delta_x.$$

- Step 2 — T^F is linear and sends $\delta_x \mapsto \delta_{F(x)}$:

$$T^F \beta_u = \sum_{x \in X} \beta_u[x] \delta_{F(x)}.$$

- Step 3 — apply the linear functional β^v :

$$A^F[v, u] = \beta^v(T^F \beta_u) = \sum_{x \in X} \beta_u[x] \beta^v(\delta_{F(x)}) = \sum_{x \in X} \beta_u[x] \beta^v(F(x)),$$

using $\beta^v(y) := \beta^v(\delta_y)$ from the dual-basis slide.



Worked example: $X = \mathbb{F}_2$, $F = \text{swap}$

- Standard basis $\beta_u = \delta_u$, swap $F(0) = 1$, $F(1) = 0$. So $\beta_0[0] = 1$, $\beta_0[1] = 0$, $\beta^1(0) = 0$, $\beta^1(1) = 1$.



Worked example: $X = \mathbb{F}_2$, $F = \text{swap}$

- Standard basis $\beta_u = \delta_u$, swap $F(0) = 1$, $F(1) = 0$. So $\beta_0[0] = 1$, $\beta_0[1] = 0$, $\beta^1(0) = 0$, $\beta^1(1) = 1$.
- One entry, $A^F[1, 0] = \sum_{x \in \{0,1\}} \beta_0[x] \beta^1(F(x))$:

$$\beta_0[0] \beta^1(F(0)) + \beta_0[1] \beta^1(F(1)) = \beta_0[0] \beta^1(1) + \beta_0[1] \beta^1(0) = 1 \cdot 1 + 0 \cdot 0 = 1.$$



Worked example: $X = \mathbb{F}_2$, $F = \text{swap}$

- Standard basis $\beta_u = \delta_u$, swap $F(0) = 1$, $F(1) = 0$. So $\beta_0[0] = 1$, $\beta_0[1] = 0$, $\beta^1(0) = 0$, $\beta^1(1) = 1$.
- One entry, $A^F[1, 0] = \sum_{x \in \{0,1\}} \beta_0[x] \beta^1(F(x))$:

$$\beta_0[0] \beta^1(F(0)) + \beta_0[1] \beta^1(F(1)) = \beta_0[0] \beta^1(1) + \beta_0[1] \beta^1(0) = 1 \cdot 1 + 0 \cdot 0 = 1.$$

- All four entries:

$$A^F = \begin{bmatrix} A^F[0, 0] & A^F[0, 1] \\ A^F[1, 0] & A^F[1, 1] \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Matrix representation

- Once we fix a basis for $\mathbb{K}[X]$,

$$T^F : \mathbb{K}[X] \rightarrow \mathbb{K}[X]$$

has a unique corresponding matrix, called **transition matrix**: it is $A^F[v, u] = \beta^v(T^F \beta_u)$ from the previous slide.

- Under the standard basis $B = (\delta_x, x \in X)$,

$$T^F[v, u] = \delta_v(F(u)) := \begin{cases} 1 & v = F(u) \\ 0 & v \neq F(u) \end{cases}$$

Example: T^F for PRESENT's Sbox (1)

- PRESENT's Sbox is a permutation $S : \mathbb{F}_2^4 \rightarrow \mathbb{F}_2^4$,

Input x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
Output $S(x)$	c	5	6	b	9	0	a	d	3	e	f	8	4	7	1	2

$$T^S = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Example: T^F for PRESENT's Sbox (2)

$$T^S \delta_0 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{1} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbf{1} \end{bmatrix} = \delta_c$$

Why change the basis?

- Under the standard basis, T^F is just a **permutation matrix**: each entry is 0 or 1 and only records “ F maps u to v ”.
- All the information is there, but **no cryptanalytic structure is visible**.

Why change the basis?

- Under the standard basis, T^F is just a **permutation matrix**: each entry is 0 or 1 and only records “ F maps u to v ”.
 - All the information is there, but **no cryptanalytic structure is visible**.
- The operator T^F does not change; only its **matrix** does, once we fix another basis. Under a well-chosen basis, a single entry becomes a quantity we already care about.

Why change the basis?

- Under the standard basis, T^F is just a **permutation matrix**: each entry is 0 or 1 and only records “ F maps u to v ”.
 - All the information is there, but **no cryptanalytic structure is visible**.
- The operator T^F does not change; only its **matrix** does, once we fix another basis. Under a well-chosen basis, a single entry becomes a quantity we already care about.

⇒ **Choosing a basis = choosing a type of cryptanalysis.**

[Hu et al., CRYPTO 2025]

Basis and the change-of-basis matrix

- A new basis $(\beta_x, x \in X)$ of $\mathbb{K}[X]$, written against the standard basis:

$$(\beta_x, x \in X) = (\delta_x, x \in X) H$$

Basis and the change-of-basis matrix

- A new basis $(\beta_x, x \in X)$ of $\mathbb{K}[X]$, written against the standard basis:

$$(\beta_x, x \in X) = (\delta_x, x \in X) H$$

- Column by column, this says

$$\beta_x = \sum_{y \in X} H_{y,x} \delta_y,$$

i.e. the x -th column of H collects the coordinates of β_x under the standard basis.

Basis and the change-of-basis matrix

- A new basis $(\beta_x, x \in X)$ of $\mathbb{K}[X]$, written against the standard basis:

$$(\beta_x, x \in X) = (\delta_x, x \in X) H$$

- Column by column, this says

$$\beta_x = \sum_{y \in X} H_{y,x} \delta_y,$$

i.e. the x -th column of H collects the coordinates of β_x under the standard basis.

- Since $(\delta_x, x \in X)$ is the identity matrix, we may simply write $H = (\beta_x, x \in X)$.

Basis and the change-of-basis matrix

- A new basis $(\beta_x, x \in X)$ of $\mathbb{K}[X]$, written against the standard basis:

$$(\beta_x, x \in X) = (\delta_x, x \in X) H$$

- Column by column, this says

$$\beta_x = \sum_{y \in X} H_{y,x} \delta_y,$$

i.e. the x -th column of H collects the coordinates of β_x under the standard basis.

- Since $(\delta_x, x \in X)$ is the identity matrix, we may simply write $H = (\beta_x, x \in X)$.
- $(\beta_x, x \in X)$ is a basis $\iff H$ is invertible.

A small example: $X = \mathbb{F}_2$

- $\mathbb{K}[X]$ is 2-dimensional with standard basis δ_0, δ_1 . Take

$$\beta_0 = \delta_0 + \delta_1, \quad \beta_1 = \delta_0 - \delta_1.$$

A small example: $X = \mathbb{F}_2$

- $\mathbb{K}[X]$ is 2-dimensional with standard basis δ_0, δ_1 . Take

$$\beta_0 = \delta_0 + \delta_1, \quad \beta_1 = \delta_0 - \delta_1.$$

- Collecting the coordinates column-wise,

$$H = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H^{-1} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2}H.$$

A small example: $X = \mathbb{F}_2$

- $\mathbb{K}[X]$ is 2-dimensional with standard basis δ_0, δ_1 . Take

$$\beta_0 = \delta_0 + \delta_1, \quad \beta_1 = \delta_0 - \delta_1.$$

- Collecting the coordinates column-wise,

$$H = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H^{-1} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2}H.$$

- Note $\beta_u = [(-1)^{u^\top x}, x \in \mathbb{F}_2]$: this is already the **linear basis**, here for $n = 1$.

How coordinates transform

- Let $w \in \mathbb{K}[X]$ have coordinates u under the standard basis, i.e.
 $w = (\delta_x, x \in X) u$.

How coordinates transform

- Let $w \in \mathbb{K}[X]$ have coordinates u under the standard basis, i.e.
 $w = (\delta_x, x \in X) u$.
- Insert $HH^{-1} = I$:

$$w = (\delta_x, x \in X) HH^{-1}u = (\beta_x, x \in X) (H^{-1}u).$$

So the coordinates of w under the new basis are

$$u' = H^{-1}u.$$

How coordinates transform

- Let $w \in \mathbb{K}[X]$ have coordinates u under the standard basis, i.e.
 $w = (\delta_x, x \in X) u$.
- Insert $HH^{-1} = I$:

$$w = (\delta_x, x \in X) HH^{-1}u = (\beta_x, x \in X) (H^{-1}u).$$

So the coordinates of w under the new basis are

$$u' = H^{-1}u.$$

! The **basis vectors** are transformed by H , the **coordinates** by H^{-1} — opposite directions. Mixing these two up is the most common source of confusion here.

How the matrix transforms

- Given $T^F u = v$ under the standard basis, substitute $u = Hu'$ and $v = Hv'$:

$$T^F Hu' = Hv' \implies (H^{-1}T^F H)u' = v'.$$

How the matrix transforms

- Given $T^F u = v$ under the standard basis, substitute $u = Hu'$ and $v = Hv'$:

$$T^F Hu' = Hv' \implies (H^{-1}T^F H)u' = v'.$$

- Transition matrix under the new basis:

$$A^F = H^{-1}T^F H.$$

How the matrix transforms

- Given $T^F u = v$ under the standard basis, substitute $u = Hu'$ and $v = Hv'$:

$$T^F Hu' = Hv' \implies (H^{-1}T^F H)u' = v'.$$

- Transition matrix under the new basis:

$$A^F = H^{-1}T^F H.$$

- Entry-wise this is exactly the dual-basis reading from before:

$$A^F[v, u] = \beta^v(T^F \beta_u),$$

since the v -th row of H^{-1} is β^v and the u -th column of H is β_u .

A coordinate of A^F in closed form

- Reading off one entry of $A^F = H^{-1}T^FH$: the v -th row of H^{-1} is β^v , the u -th column of H is β_u :

$$\begin{aligned}A^F[v, u] &= [\beta^v(a), a \in \mathbb{F}_2^n]^\top \cdot T^F \cdot [\beta_u[a], a \in \mathbb{F}_2^n] \\&= \left[\sum_a \beta^v(a) T_{a,0}^F, \sum_a \beta^v(a) T_{a,1}^F, \dots, \sum_a \beta^v(a) T_{a,2^n-1}^F \right]^\top \\&\quad \cdot [\beta_u[a], a \in \mathbb{F}_2^n] \\&= [\beta^v(F(x)), x \in \mathbb{F}_2^n]^\top \cdot [\beta_u[a], a \in \mathbb{F}_2^n] \\&= \sum_{x \in \mathbb{F}_2^n} \beta^v(F(x)) \beta_u[x]\end{aligned}$$

From matrix coordinates to distinguishers

- Fix a pair (v, u) and suppose the entry behaves differently in the two worlds:

$$A^F[v, u] = \theta \neq 0 \text{ for the cipher, } A^F[v, u] = 0 \text{ for a random permutation.}$$

From matrix coordinates to distinguishers

- Fix a pair (v, u) and suppose the entry behaves differently in the two worlds:

$A^F[v, u] = \theta \neq 0$ for the cipher, $A^F[v, u] = 0$ for a random permutation.

- Suppose moreover that $A^F[v, u]$ can be **estimated from data**: that it is the mean of N independent ± 1 observations. Then we are exactly back in the first part, and the data complexity is

$$N = \left(\frac{\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})}{\theta} \right)^2.$$

Example-1: change-of-basis with the linear basis

- For linear cryptanalysis, $\mathbb{K} := \mathbb{R}$, $X = \mathbb{F}_2^n$.
- Linear basis: $H = (\beta_u, u \in \mathbb{F}_2^n)$ where

$$\beta_u = [(-1)^{u^\top x}, x \in \mathbb{F}_2^n]$$

Then, $H^{-1} = 2^{-n}H$

$$C^F = H^{-1}T^F H$$

- We have

$$\begin{aligned} C^F[v, u] &= 2^{-n} [(-1)^{v^\top x}, x \in \mathbb{F}_2^n]^\top \cdot T^F \cdot [(-1)^{u^\top x}, x \in \mathbb{F}_2^n] \\ &= 2^{-n} [(-1)^{v^\top F(x)}, x \in \mathbb{F}_2^n]^\top \cdot [(-1)^{u^\top x}, x \in \mathbb{F}_2^n] \\ &= 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{u^\top x \oplus v^\top F(x)} \end{aligned}$$

Example-1: apply linear basis to PRESENT's Sbox

$$C^F = H^{-1} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \end{bmatrix} H$$

Linear approximation table of PRESENT's Sbox

- Linear approximation table (LAT):

$$C^F = \frac{1}{16} \begin{bmatrix} 16 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 8 & 0 & -8 & 0 & 8 & 0 & 8 \\ 0 & 0 & 4 & 4 & -4 & -4 & 0 & 0 & 4 & -4 & 8 & 0 & 0 & 8 & 4 & -4 \\ 0 & 0 & 4 & 4 & 4 & 4 & -8 & 8 & -4 & -4 & 0 & 0 & 0 & 0 & 4 & 4 \\ 0 & 0 & -4 & 4 & -4 & -4 & 0 & 8 & 0 & 0 & 4 & -4 & -4 & -4 & -8 & 0 \\ 0 & -8 & -4 & -4 & -4 & 4 & 0 & 0 & 0 & 0 & 4 & -4 & -4 & -4 & 8 & 0 \\ 0 & 0 & 0 & -8 & 0 & 0 & -8 & 0 & -4 & 4 & 4 & 4 & -4 & 4 & -4 & -4 \\ 0 & -8 & 0 & 0 & 8 & 0 & 0 & 0 & 4 & -4 & -4 & -4 & -4 & 4 & -4 & -4 \\ 0 & 0 & 4 & -4 & -4 & 4 & 0 & 0 & -4 & -4 & 0 & -8 & 8 & 0 & -4 & -4 \\ 0 & 0 & -4 & 4 & -4 & 4 & -8 & -8 & 4 & -4 & 0 & 0 & 0 & 0 & -4 & 4 \\ 0 & 0 & 0 & -8 & 0 & -8 & 0 & 0 & 0 & -8 & 0 & 0 & 0 & 0 & 0 & 8 \\ 0 & 0 & 8 & 0 & -8 & 0 & 0 & 0 & 0 & 0 & -8 & 0 & -8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 8 & 8 & 0 & -4 & -4 & 4 & 4 & -4 & 4 & -4 & 4 \\ 0 & -8 & 8 & 0 & 0 & 0 & 0 & 0 & 4 & 4 & 4 & 4 & 4 & -4 & -4 & 4 \\ 0 & 0 & -4 & -4 & -4 & 4 & 0 & 8 & 8 & 0 & -4 & 4 & 4 & 4 & 0 & 0 \\ 0 & 8 & 4 & -4 & 4 & 4 & 0 & 0 & 8 & 0 & 4 & -4 & -4 & -4 & 0 & 0 \end{bmatrix}$$

Contents

- 1 The task in symmetric-key cryptanalysis
- 2 Revisit linear cryptanalysis with geometric approach
- 3 Approximation with trails

Approximation

- Impossible to compute the **exact** value of a matrix coordinate (complexity is at least $\mathcal{O}(2^n)$)!
- Approximation is necessary!
- For efficient approximation, **various assumptions** are necessary!

Composition

- Under the standard basis,

For $F = F_2 \circ F_1$, where $F_i : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ for $i = 1, 2$

$$T^F = T^{F_2} \cdot T^{F_1}$$

Proof:

$$T^F \delta_x = \delta_{F(x)} = \delta_{F_2(F_1(x))} = T^{F_2} \delta_{F_1(x)} = T^{F_2} T^{F_1} \delta_x.$$

Composition

- Under the standard basis,

For $F = F_2 \circ F_1$, where $F_i : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ for $i = 1, 2$

$$T^F = T^{F_2} \cdot T^{F_1}$$

Proof:

$$T^F \delta_x = \delta_{F(x)} = \delta_{F_2(F_1(x))} = T^{F_2} \delta_{F_1(x)} = T^{F_2} T^{F_1} \delta_x.$$

- Under any basis,

A similar theorem applies to the matrix under a different basis

$$A^F = A^{F_2} \cdot A^{F_1}$$

Proof:

$$A^F = H^{-1} T^F H = (H^{-1} T^{F_2} H)(H^{-1} T^{F_1} H) = A^{F_2} \cdot A^{F_1}.$$

Parallel functions

- Under the standard basis, for $F = F_2 || F_1$,

$$T^F = T^{F_2} \otimes T^{F_1}$$

Proof:

$$T^F(\delta_x \otimes \delta_y) = \delta_{F_2(x)} \otimes \delta_{F_1(y)} = (T^{F_2} \otimes T^{F_1})(\delta_x \otimes \delta_y)$$

Parallel functions

- Under the standard basis, for $F = F_2 || F_1$,

$$T^F = T^{F_2} \otimes T^{F_1}$$

Proof:

$$T^F(\delta_x \otimes \delta_y) = \delta_{F_2(x)} \otimes \delta_{F_1(y)} = (T^{F_2} \otimes T^{F_1})(\delta_x \otimes \delta_y)$$

- Under any basis: on the product space the natural basis is $\beta_u \otimes \beta_v$, with change-of-basis matrix $H \otimes H$, so

$$A^F = (H \otimes H)^{-1} T^F (H \otimes H) = (H^{-1} \otimes H^{-1}) T^F (H \otimes H).$$

Parallel functions

- Under the standard basis, for $F = F_2 || F_1$,

$$T^F = T^{F_2} \otimes T^{F_1}$$

Proof:

$$T^F(\delta_x \otimes \delta_y) = \delta_{F_2(x)} \otimes \delta_{F_1(y)} = (T^{F_2} \otimes T^{F_1})(\delta_x \otimes \delta_y)$$

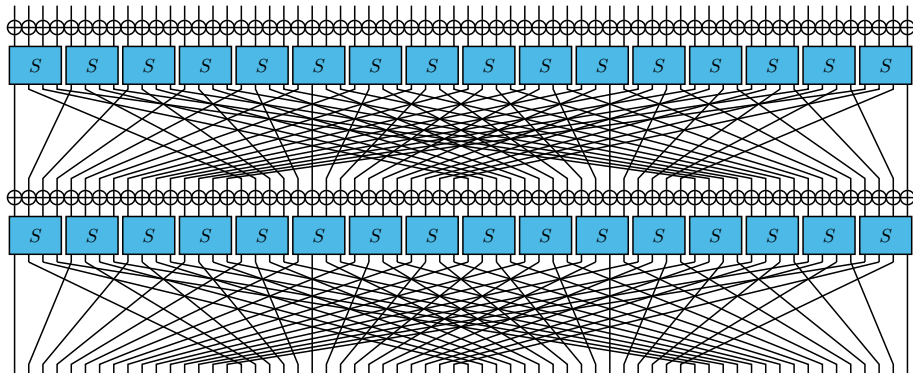
- Under any basis: on the product space the natural basis is $\beta_u \otimes \beta_v$, with change-of-basis matrix $H \otimes H$, so

$$A^F = (H \otimes H)^{-1} T^F (H \otimes H) = (H^{-1} \otimes H^{-1}) T^F (H \otimes H).$$

With this, $A^F = A^{F_2} \otimes A^{F_1}$ follows:

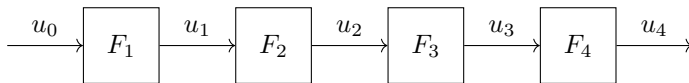
$$\begin{aligned} A^{F_2} \otimes A^{F_1} &= (H^{-1} T^{F_2} H) \otimes (H^{-1} T^{F_1} H) \\ &= (H^{-1} \otimes H^{-1}) (T^{F_2} \otimes T^{F_1}) (H \otimes H) \\ &= (H^{-1} \otimes H^{-1}) T^F (H \otimes H) = A^F. \end{aligned}$$

Apply the geometric approach to a full cipher



- Three types of components: Key-XOR, Sbox, and Linear layer.
- Construct matrices for each component.

Express matrix coordinate

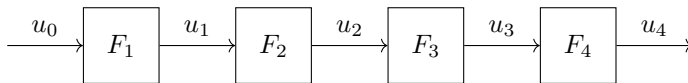


- Composition:

$$A^F = A^{F_r \circ \dots \circ F_1} = A^{F_r} \dots A^{F_2} \cdot A^{F_1}$$

$$A^F[u_r, u_0] = \sum_{u_1, \dots, u_{r-1}} A^{F_r}[u_r, u_{r-1}] \cdot A^{F_{r-1}}[u_{r-1}, u_{r-2}] \\ \dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

Express matrix coordinate



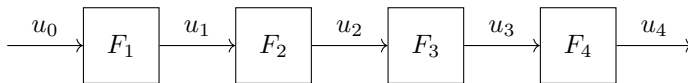
- Composition:

$$A^F = A^{F_r \circ \dots \circ F_1} = A^{F_r} \dots A^{F_2} \cdot A^{F_1}$$
$$A^F[u_r, u_0] = \sum_{u_1, \dots, u_{r-1}} A^{F_r}[u_r, u_{r-1}] \cdot A^{F_{r-1}}[u_{r-1}, u_{r-2}]$$
$$\dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

- A trail:

$$(u_0, u_1, \dots, u_r)$$

Express matrix coordinate



- Composition:

$$A^F = A^{F_r \circ \dots \circ F_1} = A^{F_r} \dots A^{F_2} \cdot A^{F_1}$$
$$A^F[u_r, u_0] = \sum_{u_1, \dots, u_{r-1}} A^{F_r}[u_r, u_{r-1}] \cdot A^{F_{r-1}}[u_{r-1}, u_{r-2}]$$
$$\dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

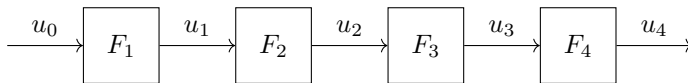
- A trail:

$$(u_0, u_1, \dots, u_r)$$

- Correlation:

$$A^{F_r}[u_r, u_{r-1}] \dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

Express matrix coordinate



- Composition:

$$A^F = A^{F_r \circ \dots \circ F_1} = A^{F_r} \dots A^{F_2} \cdot A^{F_1}$$
$$A^F[u_r, u_0] = \sum_{u_1, \dots, u_{r-1}} A^{F_r}[u_r, u_{r-1}] \cdot A^{F_{r-1}}[u_{r-1}, u_{r-2}]$$
$$\dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

- A trail:

$$(u_0, u_1, \dots, u_r)$$

- Correlation:

$$A^{F_r}[u_r, u_{r-1}] \dots A^{F_2}[u_2, u_1] \cdot A^{F_1}[u_1, u_0]$$

- Result:

$$A^F[u_r, u_0] \text{ is the sum of all trails' correlations!}$$

Approximation in linear cryptanalysis (1)

- From here on we fix the linear basis, so every matrix is written C instead of the generic A : $C^F = H^{-1}T^F H$ with H the linear basis, and $C^F[v, u]$ is a correlation.
- Linear trail: $(u_0, u_1, \dots, u_r)$ (aka linear characteristic).
- Dominant trail assumption:

$$\begin{aligned} C^F[u_r, u_0] &= \sum_{u_1, \dots, u_{r-1}} C^{F_r}[u_r, u_{r-1}] \cdot C^{F_{r-1}}[u_{r-1}, u_{r-2}] \\ &\quad \dots C^{F_2}[u_2, u_1] \cdot C^{F_1}[u_1, u_0] \\ &\approx C^{F_r}[u_r, u_{r-1}] \cdot C^{F_{r-1}}[u_{r-1}, u_{r-2}] \dots C^{F_2}[u_2, u_1] \\ &\quad \dots C^{F_1}[u_1, u_0] \end{aligned}$$

Approximation in linear cryptanalysis (2)

- In linear cryptanalysis, we assume

$$|C^F[u_r, u_0]| \approx \max_{u_1, \dots, u_{r-1}} \left| C^{F_r}[u_r, u_{r-1}] \cdot C^{F_{r-1}}[u_{r-1}, u_{r-2}] \right. \\ \left. \dots C^{F_2}[u_2, u_1] \cdot C^{F_1}[u_1, u_0] \right|$$

- Need to find out **a linear trail with the maximum absolute correlation.**

Approximation in linear cryptanalysis (3)

- Key-XOR (fixed-key): $F_k : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n ; x \mapsto x \oplus k$

$$C^{F_k}[v, u] = (-1)^{v^\top k} \delta_v(u)$$

- Sbox: $S : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$, $C^S[v, u]$ from LAT

- Linear layer:

$$C^F[v, u] = \delta_u(L^\top v)$$

- A trail:

$$\begin{array}{ccccc} u_0 & \xrightarrow{\text{Key-XOR}} & u_1 & \xrightarrow{\text{Sbox}} & u_2 & \xrightarrow{\text{Linear layer } L} & u_3 \\ \underbrace{\hspace{1.5cm}} & & \underbrace{\hspace{1.5cm}} & & \underbrace{\hspace{1.5cm}} & & \\ (-1)^{u_1^\top k, u_0=u_1} & & C^S[u_2, u_1] & & L^\top u_3=u_2 & & \end{array}$$

Finding the best trail

- The task: among all trails $(u_0, u_1, \dots, u_r)$, find one maximising $|\prod_i C^{F_i}[u_i, u_{i-1}]|$, i.e. **minimising** the weight

$$w = \sum_{i=1}^r -\log_2 |C^{F_i}[u_i, u_{i-1}]|.$$

Finding the best trail

- The task: among all trails $(u_0, u_1, \dots, u_r)$, find one maximising $|\prod_i C^{F_i}[u_i, u_{i-1}]|$, i.e. **minimising** the weight

$$w = \sum_{i=1}^r -\log_2 |C^{F_i}[u_i, u_{i-1}]|.$$

- Every constraint comes directly from what we already derived:
 - **Sbox**: the non-zero entries of the LAT are the feasible transitions, each with weight $-\log_2 |C^S[v, u]|$.
 - **Linear layer**: $u = L^\top v$; for a bit-permutation, just a rewiring.
 - **Key-XOR**: $C^{F_k}[v, u] = (-1)^{v^\top k} \delta_v(u)$ only flips a sign, so it does not change $|C|$ — the search is **key-independent**.

Result: best linear trails of round-reduced PRESENT

Solving that model for PRESENT's Sbox and bit-permutation layer:

rounds r	minimal weight	$ C $ of the best trail
1	1	2^{-1}
2	2	2^{-2}
3	4	2^{-4}
4	6	2^{-6}

Result: best linear trails of round-reduced PRESENT

Solving that model for PRESENT's Sbox and bit-permutation layer:

rounds r	minimal weight	$ C $ of the best trail
1	1	2^{-1}
2	2	2^{-2}
3	4	2^{-4}
4	6	2^{-6}

A best 4-round trail — exactly one active Sbox per round:

round	input mask	output mask	weight
1	0a00000000000000	0200000000000000	1
2	0000000040000000	0000000080000000	2
3	0080000000000000	0080000000000000	2
4	2000000000000000	b0000000000000000	1

Result: best linear trails of round-reduced PRESENT

Solving that model for PRESENT's Sbox and bit-permutation layer:

rounds r	minimal weight	$ C $ of the best trail
1	1	2^{-1}
2	2	2^{-2}
3	4	2^{-4}
4	6	2^{-6}

A best 4-round trail — **exactly one active Sbox per round**:

round	input mask	output mask	weight
1	0a00000000000000	0200000000000000	1
2	0000000040000000	0000000080000000	2
3	0080000000000000	0080000000000000	2
4	2000000000000000	b0000000000000000	1

Careful: this is the correlation of **one** trail. The true $C^F[u_r, u_0]$ is the sum over all trails — the dominant trail assumption is exactly what we are relying on.

Part I takeaway

- A distinguisher is a **number** that behaves differently for the cipher and for a random permutation. Its size θ fixes the data complexity,

$$N = \left((\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})) / \theta \right)^2.$$

Part I takeaway

- A distinguisher is a **number** that behaves differently for the cipher and for a random permutation. Its size θ fixes the data complexity,

$$N = \left((\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})) / \theta \right)^2.$$

- Lifting F to the linear operator T^F costs nothing and buys everything: matrix entries are read off by the **dual basis**, $A^F[v, u] = \beta^v(T^F \beta_u)$.

Part I takeaway

- A distinguisher is a **number** that behaves differently for the cipher and for a random permutation. Its size θ fixes the data complexity,

$$N = \left((\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2})) / \theta \right)^2.$$

- Lifting F to the linear operator T^F costs nothing and buys everything: matrix entries are read off by the **dual basis**, $A^F[v, u] = \beta^v(T^F \beta_u)$.
- The operator never changes — only its matrix does. **Choosing a basis = choosing a type of cryptanalysis**; the linear basis turns $A^F[v, u]$ into a correlation and the whole matrix into the LAT.

Part I takeaway

- A distinguisher is a **number** that behaves differently for the cipher and for a random permutation. Its size θ fixes the data complexity,

$$N = ((\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2}))/\theta)^2.$$

- Lifting F to the linear operator T^F costs nothing and buys everything: matrix entries are read off by the **dual basis**, $A^F[v, u] = \beta^v(T^F \beta_u)$.
- The operator never changes — only its matrix does. **Choosing a basis = choosing a type of cryptanalysis**; the linear basis turns $A^F[v, u]$ into a correlation and the whole matrix into the LAT.
- One entry costs $\mathcal{O}(2^n)$ exactly, so we **approximate**: composition splits it into a sum over trails, and the dominant trail is kept.

Part I takeaway

- A distinguisher is a **number** that behaves differently for the cipher and for a random permutation. Its size θ fixes the data complexity,

$$N = ((\Phi^{-1}(P_S) + \Phi^{-1}(1 - \frac{P_F}{2}))/\theta)^2.$$

- Lifting F to the linear operator T^F costs nothing and buys everything: matrix entries are read off by the **dual basis**, $A^F[v, u] = \beta^v(T^F \beta_u)$.
 - The operator never changes — only its matrix does. **Choosing a basis = choosing a type of cryptanalysis**; the linear basis turns $A^F[v, u]$ into a correlation and the whole matrix into the LAT.
 - One entry costs $\mathcal{O}(2^n)$ exactly, so we **approximate**: composition splits it into a sum over trails, and the dominant trail is kept.
- ⇒ Finding the best trail is a combinatorial optimisation problem, handed to an MILP/SAT solver.

Part II

Round-based approximation of differential-linear correlation

4 From linear to differential-linear

5 Round-based approximation

6 Large χ via S-functions

Differential-linear distinguisher

- In Part I one sample was **one message**. A **differential-linear** (DL) distinguisher uses **a pair**: fix an input difference Δ and an output mask λ , and measure

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \text{Cor}_x \left[\lambda^\top (F(x) \oplus F(x \oplus \Delta)) \right] = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\lambda^\top D_\Delta F(x)},$$

where $D_\Delta F(x) = F(x) \oplus F(x \oplus \Delta)$ is the **derivative** of F in the direction Δ .

Differential-linear distinguisher

- In Part I one sample was **one message**. A **differential-linear** (DL) distinguisher uses **a pair**: fix an input difference Δ and an output mask λ , and measure

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \text{Cor}_x[\lambda^\top (F(x) \oplus F(x \oplus \Delta))] = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\lambda^\top D_\Delta F(x)},$$

where $D_\Delta F(x) = F(x) \oplus F(x \oplus \Delta)$ is the **derivative** of F in the direction Δ .

- For a random permutation this is ≈ 0 . So it is again a statistic that separates the two worlds — and the analysis of Part I applies verbatim, now with $\theta = \text{DL}[\Delta \rightarrow \lambda]$.

[Langford, Hellman, CRYPTO 1994]

One sample is a pair: the 2-wise form

- The statistic involves x and $x \oplus \Delta$ **together**, so the object we push through the cipher is a pair. Define the **2-wise form**

$$F^{\times 2} : \mathbb{F}_2^n \times \mathbb{F}_2^n \longrightarrow \mathbb{F}_2^n \times \mathbb{F}_2^n ; \quad (x, \Delta) \longmapsto (F(x), D_{\Delta}F(x)) .$$

- The second coordinate is the **difference**, not $F(x \oplus \Delta)$: the same information, but it keeps Δ in plain sight.

One sample is a pair: the 2-wise form

- The statistic involves x and $x \oplus \Delta$ **together**, so the object we push through the cipher is a pair. Define the **2-wise form**

$$F^{\times 2} : \mathbb{F}_2^n \times \mathbb{F}_2^n \longrightarrow \mathbb{F}_2^n \times \mathbb{F}_2^n ; \quad (x, \Delta) \longmapsto (F(x), D_{\Delta}F(x)) .$$

- The second coordinate is the **difference**, not $F(x \oplus \Delta)$: the same information, but it keeps Δ in plain sight.
- The **wise** d of an attack is the number of messages in one sample:

$$d = 1 \text{ (linear, Part I)}, \quad d = 2 \text{ (DL)}, \quad d = 2^k \text{ (higher-order DL)}.$$

The matrix of $F^{\times 2}$: no new machinery

- $F^{\times 2}$ acts on $\mathbb{F}_2^n \times \mathbb{F}_2^n$, so its basis is the **Kronecker product** of two copies of the linear basis — exactly the situation of *Parallel functions* in Part I:

$$C^{F^{\times 2}} = (H \otimes H)^{-1} T^{F^{\times 2}} (H \otimes H).$$

The matrix of $F^{\times 2}$: no new machinery

- $F^{\times 2}$ acts on $\mathbb{F}_2^n \times \mathbb{F}_2^n$, so its basis is the **Kronecker product** of two copies of the linear basis — exactly the situation of *Parallel functions* in Part I:

$$C^{F^{\times 2}} = (H \otimes H)^{-1} T^{F^{\times 2}} (H \otimes H).$$

- Composition survives the lift: $(G \circ F)^{\times 2} = G^{\times 2} \circ F^{\times 2}$, hence for $E = F_r \circ \cdots \circ F_1$,

$$C^{E^{\times 2}} = C^{F_r^{\times 2}} \cdots C^{F_1^{\times 2}}.$$

The matrix of $F^{\times 2}$: no new machinery

- $F^{\times 2}$ acts on $\mathbb{F}_2^n \times \mathbb{F}_2^n$, so its basis is the **Kronecker product** of two copies of the linear basis — exactly the situation of *Parallel functions* in Part I:

$$C^{F^{\times 2}} = (H \otimes H)^{-1} T^{F^{\times 2}} (H \otimes H).$$

- Composition survives the lift: $(G \circ F)^{\times 2} = G^{\times 2} \circ F^{\times 2}$, hence for $E = F_r \circ \dots \circ F_1$,

$$C^{E^{\times 2}} = C^{F_r^{\times 2}} \dots C^{F_1^{\times 2}}.$$

⇒ We can again work **round by round** — with the same tools as Part I, only on a space of twice the dimension.

Correlation vector of a multiset

- Part I attached a correlation to a **function**. The same expression makes sense for a **multiset** S of values in $X = \mathbb{F}_2^k$, and defines the **correlation vector** $\text{CV}(S)$:

$$\text{CV}(S)[u] = \frac{1}{|S|} \sum_{x \in S} (-1)^{u^\top x}, \quad \text{CV}(S) \in \mathbb{R}[X].$$

Correlation vector of a multiset

- Part I attached a correlation to a **function**. The same expression makes sense for a **multiset** S of values in $X = \mathbb{F}_2^k$, and defines the **correlation vector** $\text{CV}(S)$:

$$\text{CV}(S)[u] = \frac{1}{|S|} \sum_{x \in S} (-1)^{u^\top x}, \quad \text{CV}(S) \in \mathbb{R}[X].$$

- $\text{CV}(S)[0] = 1$ always; the remaining coordinates measure how far S is from the uniform distribution.
 - A set of plaintexts, a set of ciphertexts, a set of differences — each has one.

The correlation vector propagates by C^F

- Let $\mathcal{C} = F(\mathcal{P})$, i.e. apply F to every element of the multiset $\mathcal{P}$. Then

$$\text{CV}(\mathcal{C}) = C^F \cdot \text{CV}(\mathcal{P}).$$

The correlation vector propagates by C^F

- Let $\mathcal{C} = F(\mathcal{P})$, i.e. apply F to every element of the multiset $\mathcal{P}$. Then

$$\text{CV}(\mathcal{C}) = C^F \cdot \text{CV}(\mathcal{P}).$$

- This is not a new theorem. It is Part I's statement that **a matrix acts on coordinates** — $c \mapsto A^F c$ — read in the linear basis.

The correlation vector propagates by C^F

- Let $\mathcal{C} = F(\mathcal{P})$, i.e. apply F to every element of the multiset $\mathcal{P}$. Then

$$CV(\mathcal{C}) = C^F \cdot CV(\mathcal{P}).$$

- This is not a new theorem. It is Part I's statement that **a matrix acts on coordinates** — $c \mapsto A^F c$ — read in the linear basis.
- ⇒ Pushing a set of plaintexts through the cipher = multiplying its correlation vector by **one correlation matrix per round**.

[Daemen, Govaerts, Vandewalle, FSE 1994] [Beyne, ASIACRYPT 2021]

DL correlation is one coordinate

- Feed in **all** messages with **one** fixed difference:

$$\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}, \quad \mathcal{C} = F^{\times 2}(\mathcal{P}).$$

DL correlation is one coordinate

- Feed in **all** messages with **one** fixed difference:

$$\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}, \quad \mathcal{C} = F^{\times 2}(\mathcal{P}).$$

- Then the $(0, \lambda)$ -coordinate of the output correlation vector is exactly the DL correlation:

$$\text{CV}(\mathcal{C})[(0, \lambda)] = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\lambda^\top D_\Delta F(x)} = \text{DL}[\Delta \xrightarrow{F} \lambda].$$

DL correlation is one coordinate

- Feed in **all** messages with **one** fixed difference:

$$\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}, \quad \mathcal{C} = F^{\times 2}(\mathcal{P}).$$

- Then the $(0, \lambda)$ -coordinate of the output correlation vector is exactly the DL correlation:

$$\text{CV}(\mathcal{C})[(0, \lambda)] = 2^{-n} \sum_{x \in \mathbb{F}_2^n} (-1)^{\lambda^\top D_{\Delta} F(x)} = \text{DL}[\Delta \xrightarrow{F} \lambda].$$

- ⇒ Computing a DL correlation = propagating **one vector** through the rounds and reading **one coordinate** at the end.

4 From linear to differential-linear

5 Round-based approximation

6 Large χ via S-functions

The wall: exact DL needs every trail

- Part I turned a correlation into a **sum over trails**, then kept the dominant one. For DL the same expansion is available and **exact**:

$$\text{DL}[\Delta \rightarrow \lambda] = \sum_{\text{all 2-wise trails}} (\text{product of round contributions}).$$

The wall: exact DL needs every trail

- Part I turned a correlation into a **sum over trails**, then kept the dominant one. For DL the same expansion is available and **exact**:

$$\text{DL}[\Delta \rightarrow \lambda] = \sum_{\text{all 2-wise trails}} (\text{product of round contributions}).$$

- But a single DL trail has a **tiny** correlation and there are enormous numbers of them; unlike the linear case, **no single trail dominates**. Full enumeration is out of reach beyond toy sizes.

The wall: exact DL needs every trail

- Part I turned a correlation into a **sum over trails**, then kept the dominant one. For DL the same expansion is available and **exact**:

$$\text{DL}[\Delta \rightarrow \lambda] = \sum_{\text{all 2-wise trails}} (\text{product of round contributions}).$$

- But a single DL trail has a **tiny** correlation and there are enormous numbers of them; unlike the linear case, **no single trail dominates**. Full enumeration is out of reach beyond toy sizes.
- ⇒ We need an approximation that **never enumerates trails** — stay inside the geometric framework, but propagate **round by round**.

Round by round: two cheap steps per round

- By $C^{E \times 2} = \prod_i C^{F_i \times 2}$ we push the correlation vector through the rounds. Split an SPN round as $F_i = L \circ S$:

$$\text{CV}(\mathcal{P}) = \sigma_0 \xrightarrow{C^{S \times 2}} \gamma_1 \xrightarrow{C^{L \times 2}} \sigma_1 \rightarrow \cdots \rightarrow \gamma_r = \text{CV}(\mathcal{C}).$$

Round by round: two cheap steps per round

- By $C^{E \times 2} = \prod_i C^{F_i \times 2}$ we push the correlation vector through the rounds. Split an SPN round as $F_i = L \circ S$:

$$\text{CV}(\mathcal{P}) = \sigma_0 \xrightarrow{C^{S \times 2}} \gamma_1 \xrightarrow{C^{L \times 2}} \sigma_1 \rightarrow \cdots \rightarrow \gamma_r = \text{CV}(\mathcal{C}).$$

- Two cheap steps per round, all in a **local** representation:
 - S-box layer**: a Kronecker product, so it acts S-box by S-box — **exact**.
 - Linear layer**: only permutes masks, but **mixes S-box positions** — here an approximation enters.

Round by round: two cheap steps per round

- By $C^{E \times 2} = \prod_i C^{F_i \times 2}$ we push the correlation vector through the rounds. Split an SPN round as $F_i = L \circ S$:

$$\text{CV}(\mathcal{P}) = \sigma_0 \xrightarrow{C^{S \times 2}} \gamma_1 \xrightarrow{C^{L \times 2}} \sigma_1 \rightarrow \cdots \rightarrow \gamma_r = \text{CV}(\mathcal{C}).$$

- Two cheap steps per round, all in a **local** representation:
 - S-box layer**: a Kronecker product, so it acts S-box by S-box — **exact**.
 - Linear layer**: only permutes masks, but **mixes S-box positions** — here an approximation enters.
- ! We never build the $2^{2n} \times 2^{2n}$ matrix; only local vectors of length 2^{2m} .

Prepare the plaintext vector σ_0

- The plaintext multiset is $\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}$. By Part I's parallel rule its correlation vector **factorizes across the S-boxes**:

$$\sigma_0 = \text{CV}(\mathcal{P}) = \bigotimes_{i=0}^{t-1} \text{CV}(\mathbb{F}_2^m \times \{\varpi_i(\Delta)\}) =: \bigotimes_{i=0}^{t-1} \sigma_0^{[i]},$$

each $\sigma_0^{[i]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

Prepare the plaintext vector σ_0

- The plaintext multiset is $\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}$. By Part I's parallel rule its correlation vector **factorizes across the S-boxes**:

$$\sigma_0 = \text{CV}(\mathcal{P}) = \bigotimes_{i=0}^{t-1} \text{CV}(\mathbb{F}_2^m \times \{\varpi_i(\Delta)\}) =: \bigotimes_{i=0}^{t-1} \sigma_0^{[i]},$$

each $\sigma_0^{[i]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

- ϖ_i is the **projection** onto the i -th S-box coordinate:

$$\varpi_i : \mathbb{F}_2^{mt} \rightarrow \mathbb{F}_2^m, \quad v \mapsto v_i.$$

Prepare the plaintext vector σ_0

- The plaintext multiset is $\mathcal{P} = \mathbb{F}_2^n \times \{\Delta\}$. By Part I's parallel rule its correlation vector **factorizes across the S-boxes**:

$$\sigma_0 = \text{CV}(\mathcal{P}) = \bigotimes_{i=0}^{t-1} \text{CV}(\mathbb{F}_2^m \times \{\varpi_i(\Delta)\}) =: \bigotimes_{i=0}^{t-1} \sigma_0^{[i]},$$

each $\sigma_0^{[i]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

- ϖ_i is the **projection** onto the i -th S-box coordinate:

$$\varpi_i : \mathbb{F}_2^{mt} \rightarrow \mathbb{F}_2^m, \quad v \mapsto v_i.$$

- Cost: only t small vectors to build, $\mathcal{O}(t \cdot 2^m)$.

Pass the S-box layer

- The S-box layer is $S = s \parallel \cdots \parallel s$ (t parallel copies), so by Part I's parallel rule

$$C^{S \times 2} = \bigotimes_{i=0}^{t-1} C^{s \times 2}.$$

Pass the S-box layer

- The S-box layer is $S = s \parallel \cdots \parallel s$ (t parallel copies), so by Part I's parallel rule

$$C^{S \times 2} = \bigotimes_{i=0}^{t-1} C^{s \times 2}.$$

- Hence applying it **keeps the Kronecker form**, acting S-box by S-box:

$$\gamma_1 = C^{S \times 2} \sigma_0 = \left(\bigotimes_i C^{s \times 2} \right) \left(\bigotimes_i \sigma_0^{[i]} \right) = \bigotimes_i \left(C^{s \times 2} \sigma_0^{[i]} \right),$$

i.e. each $\gamma_1^{[i]} = C^{s \times 2} \sigma_0^{[i]}$ is computed independently.

Pass the S-box layer

- The S-box layer is $S = s \parallel \cdots \parallel s$ (t parallel copies), so by Part I's parallel rule

$$C^{S \times 2} = \bigotimes_{i=0}^{t-1} C^{s \times 2}.$$

- Hence applying it **keeps the Kronecker form**, acting S-box by S-box:

$$\gamma_1 = C^{S \times 2} \sigma_0 = \left(\bigotimes_i C^{s \times 2} \right) \left(\bigotimes_i \sigma_0^{[i]} \right) = \bigotimes_i \left(C^{s \times 2} \sigma_0^{[i]} \right),$$

i.e. each $\gamma_1^{[i]} = C^{s \times 2} \sigma_0^{[i]}$ is computed independently.

- Local and **exact** — no error yet. Multiplying by one $C^{s \times 2}$ fast is the key trick, next.

The S-box step factorizes — the key trick

- A 2-wise S-box entry is a **product of two ordinary LAT entries**:

$$C^{\mathbf{s} \times 2}[(v_0, v_1), (u_0, u_1)] = C^{\mathbf{s}}[v_0 \oplus v_1, u_0 \oplus u_1] \cdot C^{\mathbf{s}}[v_1, u_1].$$

The S-box step factorizes — the key trick

- A 2-wise S-box entry is a **product of two ordinary LAT entries**:

$$C^{s \times 2}[(v_0, v_1), (u_0, u_1)] = C^s[v_0 \oplus v_1, u_0 \oplus u_1] \cdot C^s[v_1, u_1].$$

- So **no new table is needed** — the LAT of Part I already contains everything.

The S-box step factorizes — the key trick

- A 2-wise S-box entry is a **product of two ordinary LAT entries**:

$$C^{s^{\times 2}}[(v_0, v_1), (u_0, u_1)] = C^s[v_0 \oplus v_1, u_0 \oplus u_1] \cdot C^s[v_1, u_1].$$

- So **no new table is needed** — the LAT of Part I already contains everything.
- The factorization also turns $\gamma^{[i]} = C^{s^{\times 2}} \sigma^{[i]}$ into a **two-step contraction** (sum over u_0 , then over u_1): the cost of a whole layer drops from 2^{4m} to $\mathcal{O}(t \cdot 2^{3m})$.

Pass the linear layer

- A linear layer acts on correlation vectors by **mask transpose**:

$$\sigma_1[(v_0, v_1)] = \gamma_1[(L^{\times 2})^\top(v_0, v_1)] = \gamma_1[(L^\top v_0, L^\top v_1)].$$

Pass the linear layer

- A linear layer acts on correlation vectors by **mask transpose**:

$$\sigma_1[(v_0, v_1)] = \gamma_1[(L^{\times 2})^\top (v_0, v_1)] = \gamma_1[(L^\top v_0, L^\top v_1)].$$

- Reading $\gamma_1 = \bigotimes_j \gamma_1^{[j]}$ at this permuted mask means **projecting back to each S-box**:

$$\sigma_1[(v_0, v_1)] = \prod_{j=0}^{t-1} \gamma_1^{[j]}[(\varpi_j(L^\top v_0), \varpi_j(L^\top v_1))].$$

Pass the linear layer

- A linear layer acts on correlation vectors by **mask transpose**:

$$\sigma_1[(v_0, v_1)] = \gamma_1[(L^{\times 2})^\top(v_0, v_1)] = \gamma_1[(L^\top v_0, L^\top v_1)].$$

- Reading $\gamma_1 = \bigotimes_j \gamma_1^{[j]}$ at this permuted mask means **projecting back to each S-box**:

$$\sigma_1[(v_0, v_1)] = \prod_{j=0}^{t-1} \gamma_1^{[j]}[(\varpi_j(L^\top v_0), \varpi_j(L^\top v_1))].$$

- The result is generally **not** a Kronecker product across S-boxes anymore — this **mixing** is exactly what forces the approximation on the next slide.

The linear step: the only approximation

- The mixed result is generally **not** a Kronecker product. We force it to be one — a **rank-1 approximation**:

$$\sigma_i \approx \bigotimes_{j=0}^{t-1} \sigma_i^{[j]}.$$

The linear step: the only approximation

- The mixed result is generally **not** a Kronecker product. We force it to be one — a **rank-1 approximation**:

$$\sigma_i \approx \bigotimes_{j=0}^{t-1} \sigma_i^{[j]}.$$

⇒ Only the t local marginals $\sigma_i^{[j]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$ are kept, i.e. $\mathcal{O}(t 2^{2m})$ memory.
This is the only source of error — it says the S-boxes behave **independently**, and here that assumption is written down explicitly rather than hidden.

The whole algorithm

Input (Δ, λ) ; keep local vectors $\sigma^{[j]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

➊ **Init:** $\sigma_0^{[j]} = \text{CV}(\mathbb{F}_2^m \times \{\Delta^{[j]}\})$.

The whole algorithm

Input (Δ, λ) ; keep local vectors $\sigma^{[j]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

① **Init:** $\sigma_0^{[j]} = \text{CV}(\mathbb{F}_2^m \times \{\Delta^{[j]}\})$.

② **Each round** $i = 1, \dots, r$:

(a) S-box: $\gamma_i^{[j]} = C^{s \times 2} \sigma_{i-1}^{[j]}$ (exact)

(b) Linear: rank-1 re-factorization of $(L^{\times 2})^\top$ (approximate)

(c) Key / constant XOR: a sign $(-1)^{v_0^\top k}$ (next slide)

The whole algorithm

Input (Δ, λ) ; keep local vectors $\sigma^{[j]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

① **Init:** $\sigma_0^{[j]} = \text{CV}(\mathbb{F}_2^m \times \{\Delta^{[j]}\})$.

② **Each round** $i = 1, \dots, r$:

(a) S-box: $\gamma_i^{[j]} = C^{s \times 2} \sigma_{i-1}^{[j]}$ (exact)

(b) Linear: rank-1 re-factorization of $(L^{\times 2})^\top$ (approximate)

(c) Key / constant XOR: a sign $(-1)^{v_0^\top k}$ (next slide)

③ **Read off:** $\text{DL} = \prod_j \sigma_r^{[j]}[(0, \lambda^{[j]})]$.

The whole algorithm

Input (Δ, λ) ; keep local vectors $\sigma^{[j]} \in \mathbb{R}[\mathbb{F}_2^{2m}]$.

❶ **Init:** $\sigma_0^{[j]} = \text{CV}(\mathbb{F}_2^m \times \{\Delta^{[j]}\})$.

❷ **Each round** $i = 1, \dots, r$:

(a) S-box: $\gamma_i^{[j]} = C^{s \times 2} \sigma_{i-1}^{[j]}$ (exact)

(b) Linear: rank-1 re-factorization of $(L^{\times 2})^\top$ (approximate)

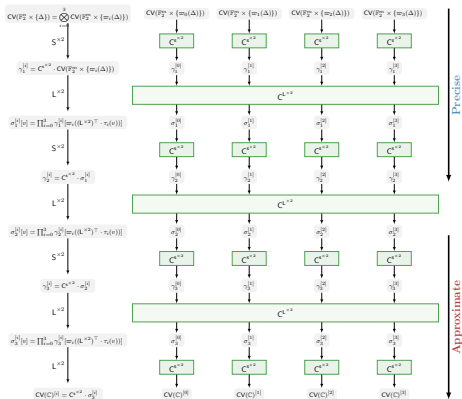
(c) Key / constant XOR: a sign $(-1)^{v_0^\top k}$ (next slide)

❸ **Read off:** $\text{DL} = \prod_j \sigma_r^{[j]}[(0, \lambda^{[j]})]$.

Cost $\mathcal{O}(t 2^{3m})$ per round; for $m = 4, 5$ this runs in **milliseconds**, with **no trail enumeration at all**.

The pipeline at a glance

Local vectors $\sigma^{[j]}, \gamma^{[j]}$ move down; each round applies $C^{s \times 2}$ and then $C^{L \times 2}$. Round 1 is **exact**; the rank-1 error enters from round 2 on.



Controlling the error: extend, then approximate

- The rank-1 assumption only bites once the difference has **spread** across S-boxes. So treat the early rounds differently. Split $F = F_2 \circ F_1$:

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \sum_{\Delta'} p_{\Delta'} \cdot \text{DL}[\Delta' \xrightarrow{F_2} \lambda], \quad p_{\Delta'} = \Pr_x [D_{\Delta} F_1(x) = \Delta'].$$

Controlling the error: extend, then approximate

- The rank-1 assumption only bites once the difference has **spread** across S-boxes. So treat the early rounds differently. Split $F = F_2 \circ F_1$:

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \sum_{\Delta'} p_{\Delta'} \cdot \text{DL}[\Delta' \xrightarrow{F_2} \lambda], \quad p_{\Delta'} = \Pr_x [D_{\Delta} F_1(x) = \Delta'].$$

- First r_1 rounds: only track **which differences Δ' occur and how often** — no output masks yet, so the vector stays **exactly** Kronecker-factored.

Controlling the error: extend, then approximate

- The rank-1 assumption only bites once the difference has **spread** across S-boxes. So treat the early rounds differently. Split $F = F_2 \circ F_1$:

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \sum_{\Delta'} p_{\Delta'} \cdot \text{DL}[\Delta' \xrightarrow{F_2} \lambda], \quad p_{\Delta'} = \Pr_x [D_{\Delta} F_1(x) = \Delta'].$$

- First r_1 rounds: only track **which differences Δ' occur and how often** — no output masks yet, so the vector stays **exactly** Kronecker-factored.
- Remaining rounds: round-based approximation started from each Δ' .

Controlling the error: extend, then approximate

- The rank-1 assumption only bites once the difference has **spread** across S-boxes. So treat the early rounds differently. Split $F = F_2 \circ F_1$:

$$\text{DL}[\Delta \xrightarrow{F} \lambda] = \sum_{\Delta'} p_{\Delta'} \cdot \text{DL}[\Delta' \xrightarrow{F_2} \lambda], \quad p_{\Delta'} = \Pr_x [D_{\Delta} F_1(x) = \Delta'].$$

- First r_1 rounds: only track **which differences Δ' occur and how often** — no output masks yet, so the vector stays **exactly** Kronecker-factored.
 - Remaining rounds: round-based approximation started from each Δ' .
- ⇒ Postponing the moment the assumption breaks buys **longer and tighter** distinguishers.

Key-XOR is just a sign flip

- For $K_k : (x, \theta) \mapsto (x \oplus k, \theta)$ the 2-wise correlation matrix is **diagonal**:

$$C^{K_k^{\times 2}}[(v_0, v_1), (u_0, u_1)] = (-1)^{v_0^\top k} \delta_{u_0}(v_0) \delta_{u_1}(v_1).$$

XORing k only multiplies coordinate (v_0, v_1) by a sign depending on the **value mask** v_0 .

Key-XOR is just a sign flip

- For $K_k : (x, \theta) \mapsto (x \oplus k, \theta)$ the 2-wise correlation matrix is **diagonal**:

$$C^{K_k^{\times 2}}[(v_0, v_1), (u_0, u_1)] = (-1)^{v_0^\top k} \delta_{u_0}(v_0) \delta_{u_1}(v_1).$$

XORing k only multiplies coordinate (v_0, v_1) by a sign depending on the **value mask** v_0 .

- Fixed key** (or permutation constants): keep the exact sign — the estimate is for the **actual** instance, not an average.

Key-XOR is just a sign flip

- For $K_k : (x, \theta) \mapsto (x \oplus k, \theta)$ the 2-wise correlation matrix is **diagonal**:

$$C^{K_k^{\times 2}}[(v_0, v_1), (u_0, u_1)] = (-1)^{v_0^\top k} \delta_{u_0}(v_0) \delta_{u_1}(v_1).$$

XORing k only multiplies coordinate (v_0, v_1) by a sign depending on the **value mask** v_0 .

- Fixed key** (or permutation constants): keep the exact sign — the estimate is for the **actual** instance, not an average.
- Average key**: k uniform $\Rightarrow$ every coordinate with $v_0 \neq 0$ vanishes, so one may set $v_0 = 0$ safely.

Average key: why the value mask must vanish

- Under **average key**, k is uniform in $\mathbb{F}_2^n$, so the sign $(-1)^{v_0^\top k}$ is averaged over all keys:

$$\mathbb{E}_k [(-1)^{v_0^\top k}] = 2^{-n} \sum_{k \in \mathbb{F}_2^n} (-1)^{v_0^\top k}.$$

Average key: why the value mask must vanish

- Under **average key**, k is uniform in $\mathbb{F}_2^n$, so the sign $(-1)^{v_0^\top k}$ is averaged over all keys:

$$\mathbb{E}_k [(-1)^{v_0^\top k}] = 2^{-n} \sum_{k \in \mathbb{F}_2^n} (-1)^{v_0^\top k}.$$

- For $u \neq 0$, the linear form $k \mapsto u^\top k$ takes the values 0 and 1 **each on exactly 2^{n-1} points**, so the terms cancel:

$$\sum_{k \in \mathbb{F}_2^n} (-1)^{u^\top k} = 2^{n-1} \cdot (+1) + 2^{n-1} \cdot (-1) = 0,$$

while for $u = 0$ all 2^n terms are 1.

Average key: why the value mask must vanish

- Under **average key**, k is uniform in $\mathbb{F}_2^n$, so the sign $(-1)^{v_0^\top k}$ is averaged over all keys:

$$\mathbb{E}_k [(-1)^{v_0^\top k}] = 2^{-n} \sum_{k \in \mathbb{F}_2^n} (-1)^{v_0^\top k}.$$

- For $u \neq 0$, the linear form $k \mapsto u^\top k$ takes the values 0 and 1 **each on exactly 2^{n-1} points**, so the terms cancel:

$$\sum_{k \in \mathbb{F}_2^n} (-1)^{u^\top k} = 2^{n-1} \cdot (+1) + 2^{n-1} \cdot (-1) = 0,$$

while for $u = 0$ all 2^n terms are 1.

- Hence

$$\mathbb{E}_k [(-1)^{v_0^\top k}] = \begin{cases} 1 & v_0 = 0, \\ 0 & v_0 \neq 0, \end{cases}$$

i.e. every coordinate with $v_0 \neq 0$ **vanishes** — only $v_0 = 0$ survives.

ASCON: most precise, and one more round

ASCON is a sponge-based design whose permutation is an SPN with small S-boxes, so the pipeline above applies to it unchanged.

Target	Rounds	Th. Cor.	Method
ASCON-128 init.	4	2^{-1}	DLCT / HATF / TDT
	4	2^{-1}	round-based
	5	$2^{-9.1}$	TDT
	5	$2^{-8.94}$	round-based (matches experiment)
ASCON-128a init.	6	$2^{-23.89}$	round-based (new)
ASCON- <i>p</i>	5	$2^{-4.0} / 2^{-6.83}$	GDLCT
	5	$2^{-4.21} / 2^{-7.58}$	round-based
	6	$-2^{-21.89}$	round-based (new)

Theory follows experiment closely, and reaches **one more round** than every earlier method.

PRESENT: from 13 to 17 rounds

Target	Rounds	Th. Cor.	Method
PRESENT	13	$2^{-27.01}$	GDLCT
PRESENT	13	$2^{-22.43}$	round-based
PRESENT	17	$2^{-29.76}$	round-based (new)

- At 13 rounds the correlation is **much stronger** than the earlier estimate — the earlier method was losing a lot.
- Valid DL distinguishers go from 13 to **17 rounds**: a 4-round jump on the cipher we used throughout Part I.

Higher-order DL: same method, wider tuples

- Replace pairs by 2^d -tuples — the 2^d -wise form. Nothing in the pipeline changes; only the local alphabet grows.

Higher-order DL: same method, wider tuples

- Replace pairs by 2^d -tuples — the 2^d -wise form. Nothing in the pipeline changes; only the local alphabet grows.
- For an SPN with t parallel m -bit S-boxes, a d -th order DL evaluation costs

$$\mathcal{O}(t \cdot 2^{(2^d+1)m}).$$

Higher-order DL: same method, wider tuples

- Replace pairs by 2^d -tuples — the 2^d -wise form. Nothing in the pipeline changes; only the local alphabet grows.
- For an SPN with t parallel m -bit S-boxes, a d -th order DL evaluation costs

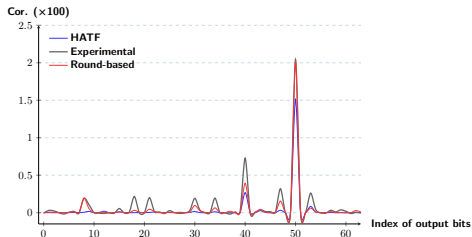
$$\mathcal{O}(t \cdot 2^{(2^d+1)m}).$$

- So $d = 2$ (quartets) is comfortably in reach for $m = 4, 5$, while $d = 3$ is already expensive.

Second-order DL on ASCON-128

Rounds	Exp.	Th. Cor.	Method
5	$2^{-5.60}$	$2^{-6.05}$	HATF
		$2^{-5.63}$	round-based
6	—	$2^{-45.35}$	round-based (new)

At 5 rounds the round-based estimate is **closer** to the experiment than HATF's; and it gives the first valid second-order DL on **6-round** initialization.



5 rounds, all 64 single-bit output masks: round-based tracks the experiment and beats HATF.

- DL correlation = the $(0, \lambda)$ coordinate of the ciphertext correlation vector; propagate that vector round by round.
- S-box step **exact** (a product of two LAT entries); linear step a **rank-1** approximation — the single assumption.
- $\mathcal{O}(t 2^{3m})$ per round, no trail enumeration; best known DL / higher-order DL for ASCON and PRESENT; key handled exactly as a sign.

Where we are

- DL correlation = the $(0, \lambda)$ coordinate of the ciphertext correlation vector; propagate that vector round by round.
- S-box step **exact** (a product of two LAT entries); linear step a **rank-1** approximation — the single assumption.
- $\mathcal{O}(t 2^{3m})$ per round, no trail enumeration; best known DL / higher-order DL for ASCON and PRESENT; key handled exactly as a sign.

The remaining problem: ciphers whose nonlinear layer is **one huge χ** on 257 bits. There are no small S-boxes to factor over — the local trick breaks down.

4 From linear to differential-linear

5 Round-based approximation

6 Large χ via S-functions

The problem: one huge χ

- SUBTERRANEAN 2.0 and KOALA have a **single** nonlinear layer: χ acting on **257 bits** — not a row of small S-boxes.

The problem: one huge χ

- SUBTERRANEAN 2.0 and KOALA have a **single** nonlinear layer: χ acting on **257 bits** — not a row of small S-boxes.
- Everything in the previous section factored over small S-boxes. With one 257-bit χ there is nothing to factor over, and we would need the correlation matrix of χ — and of $\chi^{\times 2}$ — as a whole. Even though χ is quadratic, that matrix has 2^{514} entries.

The problem: one huge χ

- SUBTERRANEAN 2.0 and KOALA have a **single** nonlinear layer: χ acting on **257 bits** — not a row of small S-boxes.
 - Everything in the previous section factored over small S-boxes. With one 257-bit χ there is nothing to factor over, and we would need the correlation matrix of χ — and of $\chi^{\times 2}$ — as a whole. Even though χ is quadratic, that matrix has 2^{514} entries.
- ⇒ **Key idea:** χ is a **Mealy machine** with a two-bit state. Its correlation matrix entries are products of **constant-size** matrices, so we never write the big matrix down.

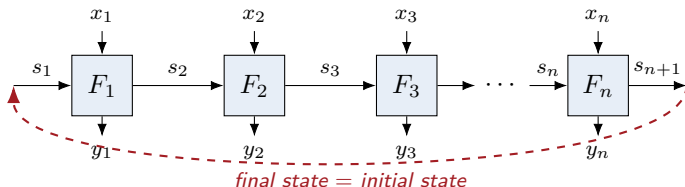
[Niu, Beyne, Hu, Wang, *Cryptanalytic Properties of Mealy Machines*]

χ is a Mealy machine (S-function)

A **Mealy machine** reads input chunks x_i , writes output chunks y_i , and carries a small **state** s_i between them. For χ ,

$$y_i = x_i \oplus (x_{i-1} \oplus 1)x_{i-2} \quad (\text{indices cyclic}),$$

so each y_i depends on the **previous two** inputs: take $s_i = (x_{i-1}, x_{i-2}) \in \mathbb{F}_2^2$.



$$F_i((x_{i-1}, x_{i-2}); x_i) = ((x_i, x_{i-1}), x_i \oplus (x_{i-1} \oplus 1)x_{i-2}).$$

The state stays two bits even at $n = 257$.

Why the chain does not simply split

*For the next three slides we drop back to a **general basis**: the argument works for any of them, and the linear basis of Part I is just one choice.*

- Part I gives any property as a pair (u, v) with value $v(T^F u)$. Take both in tensor-product form, $u = u_1 \otimes \cdots \otimes u_n$ and $v = v_1 \otimes \cdots \otimes v_n$:

$$v(T^F u) = \sum_{x_1, \dots, x_n} \underbrace{u_1[x_1] \cdots u_n[x_n]}_{\text{factors over } i} \cdot \underbrace{v_1(y_1) \cdots v_n(y_n)}_{\text{coupled through the state}} .$$

Why the chain does not simply split

*For the next three slides we drop back to a **general basis**: the argument works for any of them, and the linear basis of Part I is just one choice.*

- Part I gives any property as a pair (u, v) with value $v(T^F u)$. Take both in tensor-product form, $u = u_1 \otimes \cdots \otimes u_n$ and $v = v_1 \otimes \cdots \otimes v_n$:

$$v(T^F u) = \sum_{x_1, \dots, x_n} \underbrace{u_1[x_1] \cdots u_n[x_n]}_{\text{factors over } i} \cdot \underbrace{v_1(y_1) \cdots v_n(y_n)}_{\text{coupled through the state}} .$$

- If each y_i depended only on x_i , this would split into n independent $\mathcal{O}(1)$ sums. It does not: y_i depends on x_i **and** on s_i , which is a function of the earlier inputs.

Why the chain does not simply split

For the next three slides we drop back to a *general basis*: the argument works for any of them, and the linear basis of Part I is just one choice.

- Part I gives any property as a pair (u, v) with value $v(T^F u)$. Take both in tensor-product form, $u = u_1 \otimes \cdots \otimes u_n$ and $v = v_1 \otimes \cdots \otimes v_n$:

$$v(T^F u) = \sum_{x_1, \dots, x_n} \underbrace{u_1[x_1] \cdots u_n[x_n]}_{\text{factors over } i} \cdot \underbrace{v_1(y_1) \cdots v_n(y_n)}_{\text{coupled through the state}} .$$

- If each y_i depended only on x_i , this would split into n independent $\mathcal{O}(1)$ sums. It does not: y_i depends on x_i **and** on s_i , which is a function of the earlier inputs.
- ! But the coupling between the past and the future passes **entirely through the single state s_i** . Fix s_i and the chain falls apart.

Splitting over the state: a matrix product appears

- Insert a sum over all state sequences $s_1, \dots, s_{n+1}$ allowed by F . With the states pinned down, the product **factors over i** :

$$v(T^F u) = \sum_{s_1, \dots, s_{n+1}} \prod_{i=1}^n \underbrace{\sum_{x_i, y_i} u_i[x_i] v_i(y_i) [F_i(s_i, x_i) = (s_{i+1}, y_i)]}_{=: M_i[s_{i+1}, s_i]}.$$

Splitting over the state: a matrix product appears

- Insert a sum over all state sequences $s_1, \dots, s_{n+1}$ allowed by F . With the states pinned down, the product **factors over i** :

$$v(T^F u) = \sum_{s_1, \dots, s_{n+1}} \prod_{i=1}^n \underbrace{\sum_{x_i, y_i} u_i[x_i] v_i(y_i) [F_i(s_i, x_i) = (s_{i+1}, y_i)]}_{=: M_i[s_{i+1}, s_i]}.$$

- The outer sum over states is exactly a **matrix product**, with the initial and final state encoded by the vectors a, b :

$$M_i = \begin{bmatrix} \sum_{F_i(0, x_i) = (0, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) & \sum_{F_i(1, x_i) = (0, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) \\ \sum_{F_i(0, x_i) = (1, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) & \sum_{F_i(1, x_i) = (1, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) \end{bmatrix}$$

$$x_i, y_i \in \mathbb{F}_2; \text{ rows } s_{i+1} / \text{ cols } s_i$$

Splitting over the state: a matrix product appears

- Insert a sum over all state sequences $s_1, \dots, s_{n+1}$ allowed by F . With the states pinned down, the product **factors over i** :

$$v(T^F u) = \sum_{s_1, \dots, s_{n+1}} \prod_{i=1}^n \underbrace{\sum_{x_i, y_i} u_i[x_i] v_i(y_i) [F_i(s_i, x_i) = (s_{i+1}, y_i)]}_{=: M_i[s_{i+1}, s_i]}.$$

- The outer sum over states is exactly a **matrix product**, with the initial and final state encoded by the vectors a, b :

$$M_i = \begin{bmatrix} \sum_{F_i(0, x_i)=(0, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) & \sum_{F_i(1, x_i)=(0, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) \\ \sum_{F_i(0, x_i)=(1, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) & \sum_{F_i(1, x_i)=(1, y_i)}^{x_i, y_i} u_i[x_i] v_i(y_i) \end{bmatrix}$$

$$x_i, y_i \in \mathbb{F}_2; \text{ rows } s_{i+1} / \text{ cols } s_i$$

- Each M_i has size $|S_{i+1}| \times |S_i|$ — **constant**, independent of n . It is nothing but Part I's change of basis applied to **one** Mealy step.

Choosing the basis is still choosing the attack

- The formula $v(T^F u) = b M_n \cdots M_1 a$ holds for **any** basis vectors u_i, v_i — the machinery does not care which property we are after:
 - the **linear basis** of Part I $\Rightarrow$ ordinary correlation,
 - the **2-wise basis** of this part $\Rightarrow$ differential-linear,
 - other choices of basis give the other classical attacks (not covered today).

Choosing the basis is still choosing the attack

- The formula $v(T^F u) = b M_n \cdots M_1 a$ holds for **any** basis vectors u_i, v_i — the machinery does not care which property we are after:
 - the **linear basis** of Part I $\Rightarrow$ ordinary correlation,
 - the **2-wise basis** of this part $\Rightarrow$ differential-linear,
 - other choices of basis give the other classical attacks (not covered today).
- The only requirement is that the property be **compatible with the chunking**: u and v must factor over the chunks.

Choosing the basis is still choosing the attack

- The formula $v(T^F u) = b M_n \cdots M_1 a$ holds for **any** basis vectors u_i, v_i — the machinery does not care which property we are after:
 - the **linear basis** of Part I $\Rightarrow$ ordinary correlation,
 - the **2-wise basis** of this part $\Rightarrow$ differential-linear,
 - other choices of basis give the other classical attacks (not covered today).
 - The only requirement is that the property be **compatible with the chunking**: u and v must factor over the chunks.
- $\Rightarrow$ One theorem, evaluated with different (u_i, v_i) , gives all of them — exactly the Part I slogan, now for Mealy machines.

Evaluating any property in $\mathcal{O}(n)$

Input: basis vectors $u_1, \dots, u_n, v_1, \dots, v_n$ and boundary vectors a, b . **Output:** $v(T^F u)$.

- 1 **Transfer matrices.** For $i = 1, \dots, n$: start from $M_i = 0$ and, for every (s_i, x_i) , put $(s_{i+1}, y_i) \leftarrow F_i(s_i, x_i)$ and

$$M_i[s_{i+1}, s_i] += u_i[x_i] v_i(y_i).$$

Evaluating any property in $\mathcal{O}(n)$

Input: basis vectors $u_1, \dots, u_n, v_1, \dots, v_n$ and boundary vectors a, b . **Output:** $v(T^F u)$.

- 1 **Transfer matrices.** For $i = 1, \dots, n$: start from $M_i = 0$ and, for every (s_i, x_i) , put $(s_{i+1}, y_i) \leftarrow F_i(s_i, x_i)$ and

$$M_i[s_{i+1}, s_i] += u_i[x_i] v_i(y_i).$$

- 2 **Fold the chain.** $z \leftarrow a$; for $i = 1, \dots, n$ set $z \leftarrow M_i z$; return $b(z)$.

Evaluating any property in $\mathcal{O}(n)$

Input: basis vectors $u_1, \dots, u_n, v_1, \dots, v_n$ and boundary vectors a, b . **Output:** $v(T^F u)$.

- ① **Transfer matrices.** For $i = 1, \dots, n$: start from $M_i = 0$ and, for every (s_i, x_i) , put $(s_{i+1}, y_i) \leftarrow F_i(s_i, x_i)$ and

$$M_i[s_{i+1}, s_i] += u_i[x_i] v_i(y_i).$$

- ② **Fold the chain.** $z \leftarrow a$; for $i = 1, \dots, n$ set $z \leftarrow M_i z$; return $b(z)$.

Each M_i is read straight off the truth table of F_i , and the whole evaluation is n constant-size matrix-vector products: **$\mathcal{O}(n)$ time**, optimal in bit operations.

χ is cyclic $\Rightarrow$ the product becomes a trace

- χ feeds the final state back into the initial one, so we must force them equal: take $a = \delta_s$, $b = \delta^s$ and **sum over the free state** $s \in \mathbb{F}_2^2$.

χ is cyclic $\Rightarrow$ the product becomes a trace

- χ feeds the final state back into the initial one, so we must force them equal: take $a = \delta_s$, $b = \delta^s$ and **sum over the free state** $s \in \mathbb{F}_2^2$.
- Summing $\delta^s(M\delta_s)$ over s is precisely the trace:

$$C^X[u \rightarrow v] = \sum_{s \in \mathbb{F}_2^2} \delta^s \left(\prod_i M_i \delta_s \right) = \text{Tr} \left(\prod_{i=1}^n M_i \right).$$

χ is cyclic $\Rightarrow$ the product becomes a trace

- χ feeds the final state back into the initial one, so we must force them equal: take $a = \delta_s$, $b = \delta^s$ and **sum over the free state** $s \in \mathbb{F}_2^2$.
- Summing $\delta^s(M\delta_s)$ over s is precisely the trace:

$$C^X[u \rightarrow v] = \sum_{s \in \mathbb{F}_2^2} \delta^s \left(\prod_i M_i \delta_s \right) = \text{Tr} \left(\prod_{i=1}^n M_i \right).$$

- Since all F_i are the same map, in the linear basis there are only **four** distinct matrices M_i — one for each $(u_i, v_i) \in \mathbb{F}_2 \times \mathbb{F}_2$, each 4×4 (rows and columns indexed by the state (x_{i-1}, x_{i-2})).

From χ to $\chi^{\times 2}$

- For DL we need the **2-wise** form. Nothing conceptual changes: run the same construction on pairs, i.e. on the Mealy machine $G_i = F_i \times F_i$ with state (s_i, s'_i) .

From χ to $\chi^{\times 2}$

- For DL we need the **2-wise** form. Nothing conceptual changes: run the same construction on pairs, i.e. on the Mealy machine $G_i = F_i \times F_i$ with state (s_i, s'_i) .
- The state is now four bits, so the transfer matrices are 16×16 — still constant size — and

$$C^{\chi^{\times 2}}[\cdot] = \text{Tr}\left(\prod_{i=1}^n M_i^{\times 2}\right).$$

From χ to $\chi^{\times 2}$

- For DL we need the **2-wise** form. Nothing conceptual changes: run the same construction on pairs, i.e. on the Mealy machine $G_i = F_i \times F_i$ with state (s_i, s'_i) .
- The state is now four bits, so the transfer matrices are 16×16 — still constant size — and

$$C^{\chi^{\times 2}}[\cdot] = \text{Tr}\left(\prod_{i=1}^n M_i^{\times 2}\right).$$

$\Rightarrow$ This makes $C^{\chi^{\times 2}}$ computable for a **257-bit** χ for the first time, and it replaces the bespoke recursion that earlier work needed for χ : it is one instance of the general Mealy-machine theorem.

From one entry to the whole vector

- The trace gives **one entry** of $C^{x \times 2}$. But the round-based step needs the **whole vector** moved:

$$\gamma = C^{x \times 2} \sigma, \quad \text{i.e.} \quad \gamma[v] = \sum_{u \in \mathbb{F}_2^n \times \mathbb{F}_2^n} C^{x \times 2}[v, u] \sigma[u],$$

a sum over 2^{2n} input masks — hopeless for $n = 257$ if taken literally.

From one entry to the whole vector

- The trace gives **one entry** of $C^{x \times 2}$. But the round-based step needs the **whole vector** moved:

$$\gamma = C^{x \times 2} \sigma, \quad \text{i.e.} \quad \gamma[v] = \sum_{u \in \mathbb{F}_2^n \times \mathbb{F}_2^n} C^{x \times 2}[v, u] \sigma[u],$$

a sum over 2^{2n} input masks — hopeless for $n = 257$ if taken literally.

- Use the **same rank-1 trick** as for S-box ciphers, now **per bit**: with no small S-boxes, each **single bit** plays that role,

$$\sigma = \bigotimes_{j=0}^{n-1} \sigma^{[j]}, \quad \text{i.e.} \quad \sigma[u] = \prod_{j=0}^{n-1} \sigma^{[j]}[u_j].$$

Here $u_j := (u_0[j], u_1[j])$ and $v_j := (v_0[j], v_1[j])$.

From one entry to the whole vector

- The trace gives **one entry** of $C^{x \times 2}$. But the round-based step needs the **whole vector** moved:

$$\gamma = C^{x \times 2} \sigma, \quad \text{i.e.} \quad \gamma[v] = \sum_{u \in \mathbb{F}_2^n \times \mathbb{F}_2^n} C^{x \times 2}[v, u] \sigma[u],$$

a sum over 2^{2n} input masks — hopeless for $n = 257$ if taken literally.

- Use the **same rank-1 trick** as for S-box ciphers, now **per bit**: with no small S-boxes, each **single bit** plays that role,

$$\sigma = \bigotimes_{j=0}^{n-1} \sigma^{[j]}, \quad \text{i.e.} \quad \sigma[u] = \prod_{j=0}^{n-1} \sigma^{[j]}[u_j].$$

Here $u_j := (u_0[j], u_1[j])$ and $v_j := (v_0[j], v_1[j])$.

⇒ Both factors of the sum — the trace and σ — now factor **bit by bit**. That is what makes the sum collapse.

Absorbing σ into the transfer matrices

- Write both factors per bit and exchange sum and product:

$$\begin{aligned}\gamma[v] &= \sum_u \operatorname{Tr} \left(\prod_j M_j(u_j, v_j) \right) \prod_j \sigma^{[j]}[u_j] \\ &= \operatorname{Tr} \left(\prod_j \underbrace{\sum_{u_j} \sigma^{[j]}[u_j] M_j(u_j, v_j)}_{=: \widetilde{M}_j(v_j; \sigma^{[j]})} \right)\end{aligned}$$

Legal: each u_j meets one matrix factor and one σ factor; product and trace are linear.

Absorbing σ into the transfer matrices

- Write both factors per bit and exchange sum and product:

$$\begin{aligned}\gamma[v] &= \sum_u \operatorname{Tr} \left(\prod_j M_j(u_j, v_j) \right) \prod_j \sigma^{[j]}[u_j] \\ &= \operatorname{Tr} \left(\prod_j \underbrace{\sum_{u_j} \sigma^{[j]}[u_j] M_j(u_j, v_j)}_{=: \widetilde{M}_j(v_j; \sigma^{[j]})} \right)\end{aligned}$$

Legal: each u_j meets one matrix factor and one σ factor; product and trace are linear.

- Each $\widetilde{M}_j$ is again 16×16 — **constant size** — so

$$\gamma[v] = \operatorname{Tr} \left(\widetilde{M}_{n-1} \cdots \widetilde{M}_0 \right), \quad \widetilde{M}_j = \widetilde{M}_j(v_j; \sigma^{[j]}).$$

! $\widetilde{M}_j$ is **not** intrinsic to χ : it carries the incoming $\sigma^{[j]}$, so it is **rebuilt every round**.

$\Rightarrow$ n constant-size products and one trace: $\mathcal{O}(n)$, with **no mask enumeration**.

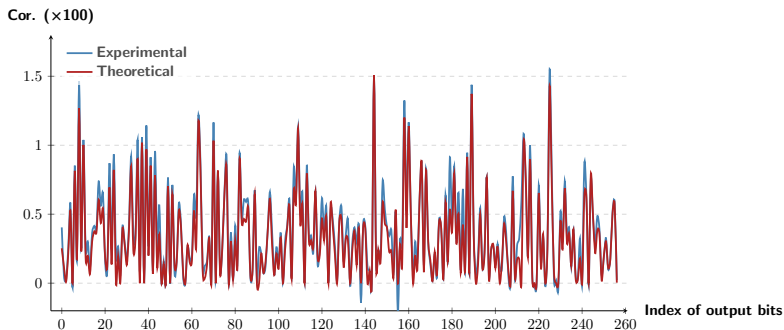
Results for SUBTERRANEAN and KOALA

Target	Rounds	Th. Cor.	Note
SUBTERRANEAN (DL)	5	$-2^{-7.98}$	exp. $-2^{-7.88}$
	6	$2^{-20.09}$	new
KOALA- p (DL)	5	$2^{-6.87}$	exp. $2^{-6.73}$
	6	$2^{-21.42}$	new
SUBTERRANEAN (2nd order)	5	$2^{-6.05}$	matches exp.
	7	$2^{-90.99}$	first
KOALA- p (2nd)	7	$2^{-86.24}$	first

DL distinguishers reach **two rounds beyond** the best differential or linear ones, and these are the first 7-round second-order DL distinguishers for both permutations.

How precise? 5-round SUBTERRANEAN, all 257 masks

Second-order DL: theory against experiment for all 257 single-bit output masks.



The two curves are visually indistinguishable — the estimate stays sharp even with one giant nonlinear layer. *KOALA-p* behaves the same way.

Part II takeaway

- A DL correlation is **one coordinate** of a correlation vector; propagating that vector round by round replaces trail enumeration entirely.
- The S-box step is exact (a product of two LAT entries); the linear step is a **rank-1** approximation — the single, explicit assumption.
- A key XOR is a **sign** on the value mask, so fixed-key estimates are sound.
- When the nonlinear layer is one huge χ , view it as a **Mealy machine**: every property becomes a product (here a trace) of constant-size matrices, evaluated in $\mathcal{O}(n)$.

Part II takeaway

- A DL correlation is **one coordinate** of a correlation vector; propagating that vector round by round replaces trail enumeration entirely.
- The S-box step is exact (a product of two LAT entries); the linear step is a **rank-1** approximation — the single, explicit assumption.
- A key XOR is a **sign** on the value mask, so fixed-key estimates are sound.
- When the nonlinear layer is one huge χ , view it as a **Mealy machine**: every property becomes a product (here a trace) of constant-size matrices, evaluated in $\mathcal{O}(n)$.

Same principle throughout: choose a basis, then read one coordinate.

Further reading

Linear cryptanalysis

- M. Matsui. *Linear Cryptanalysis Method for DES Cipher*. EUROCRYPT 1993.
- J. Daemen, R. Govaerts, J. Vandewalle. *Correlation Matrices*. FSE 1994.

Geometric approach

- T. Beyne. *A Geometric Approach to Linear Cryptanalysis*. ASIACRYPT 2021. ePrint 2021/1247
- T. Beyne. *A Geometric Approach to Symmetric-Key Cryptanalysis*. PhD thesis, KU Leuven, 2023.
- K. Hu, C. Zhang, C. Chang, J. Zhang, M. Wang, T. Peyrin. *Unlocking Mix-Basis Potential: Geometric Approach for Combined Attacks*. CRYPTO 2025.

Differential-linear and this part

- S. K. Langford, M. E. Hellman. *Differential-Linear Cryptanalysis*. CRYPTO 1994.
- K. Hu, Z. Niu, M. Wang. *Round-Based Approximation of (Higher-Order) Differential-Linear Correlation*. EUROCRYPT 2026. ePrint 2026/358
- Z. Niu, T. Beyne, K. Hu, M. Wang. *Cryptanalytic Properties of Mealy Machines*. CRYPTO 2026. ePrint 2026/1193

Cipher

- A. Bogdanov et al. *PRESENT: An Ultra-Lightweight Block Cipher*. CHES 2007.